

PowerShell

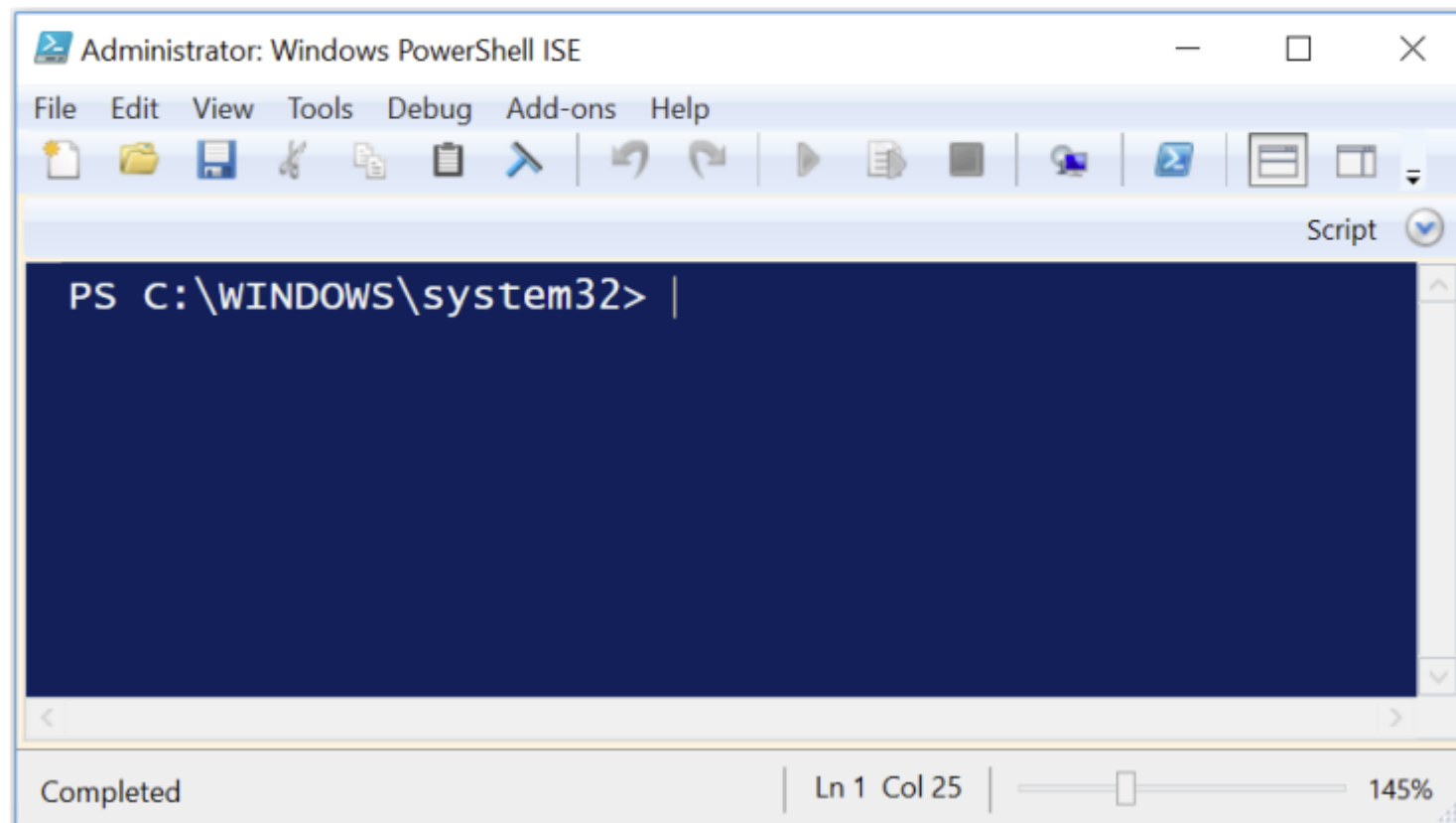
מערכות הפעלה
לינוקס

POWERSHELL

- PowerShell הוא ממשק פקודות נוסף של Windows
- ממשק זה פותח כדי לספק יכולות עשירות יותר ממש CMD שהכרנו

כדי להפעיל את Powershell, גשו לחלונת החיפוש של Windows והקלידו Powershell. תקבלו יותר מאפשרות אחת. האפשרות המומלצת היא Windows PowerShell ISE.

צפוי שהגענו אל מסך שנראה כך:

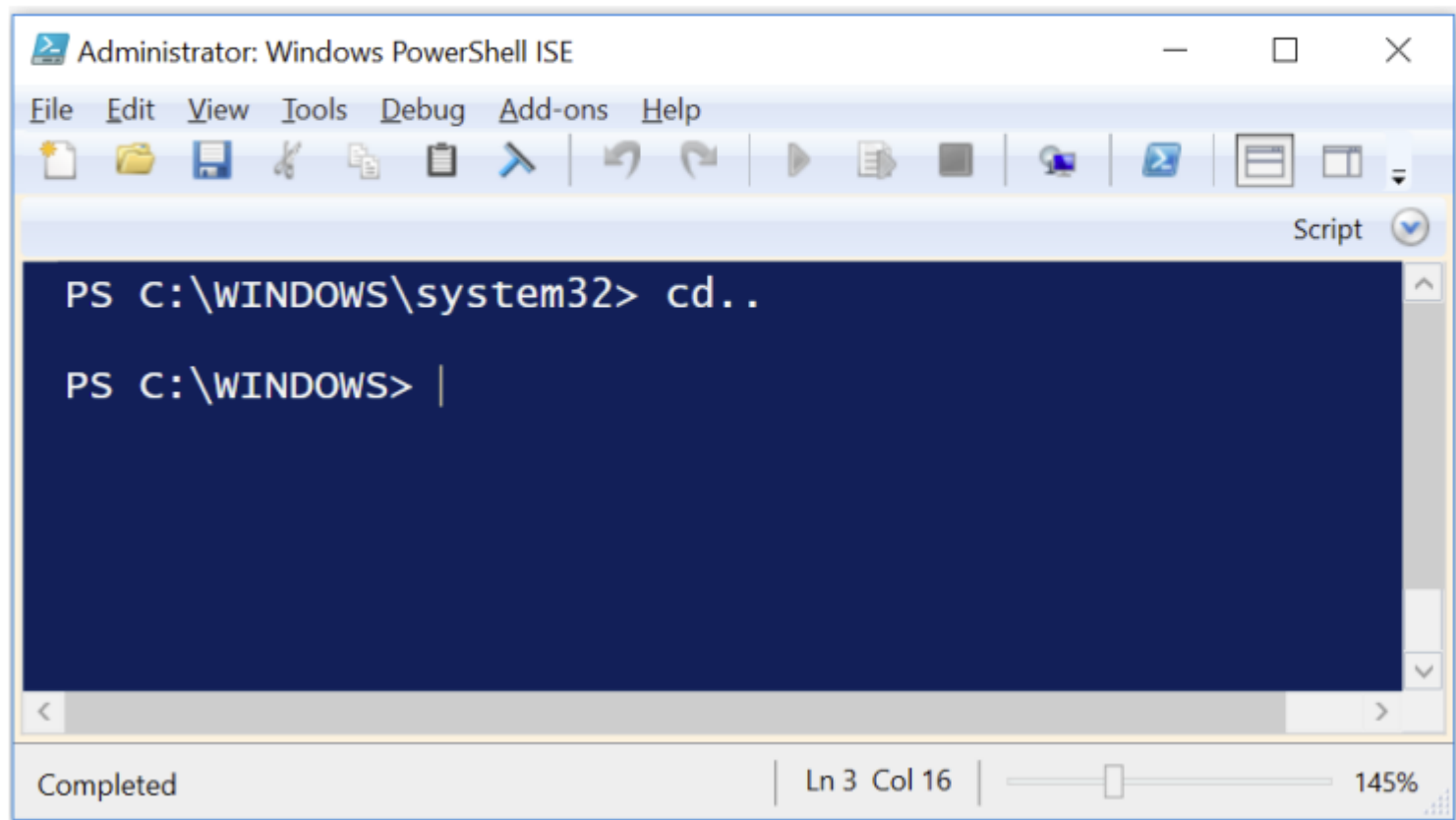


הפקודות הראשונות שנלמד הן פקודות שנלקחו בירושה מה-cmd הוותיק.

נתחיל בניווט בין תיקיות: **פקודת cd** היא קיצור של change directory. כתבו:

cd ..

מה מתרחש? נכון, התיקיה שאתם נמצאים בה השתנתה וכעת אתם נמצאים שלב אחד למטה.

A screenshot of the Windows PowerShell ISE (Integrated Scripting Environment) window. The title bar reads "Administrator: Windows PowerShell ISE". The menu bar includes "File", "Edit", "View", "Tools", "Debug", "Add-ons", and "Help". Below the menu is a toolbar with various icons for file operations and execution. The main text area has a dark blue background and shows the command prompt "PS C:\WINDOWS\system32> cd..". The second line shows the prompt "PS C:\WINDOWS> |", indicating the command was successful and the directory changed. The status bar at the bottom shows "Completed", "Ln 3 Col 16", and a zoom level of "145%".

```
Administrator: Windows PowerShell ISE
File Edit View Tools Debug Add-ons Help
PS C:\WINDOWS\system32> cd..
PS C:\WINDOWS> |
Completed | Ln 3 Col 16 | 145%
```

בואו נחזור חזרה לתיקיית system32. הקלידו

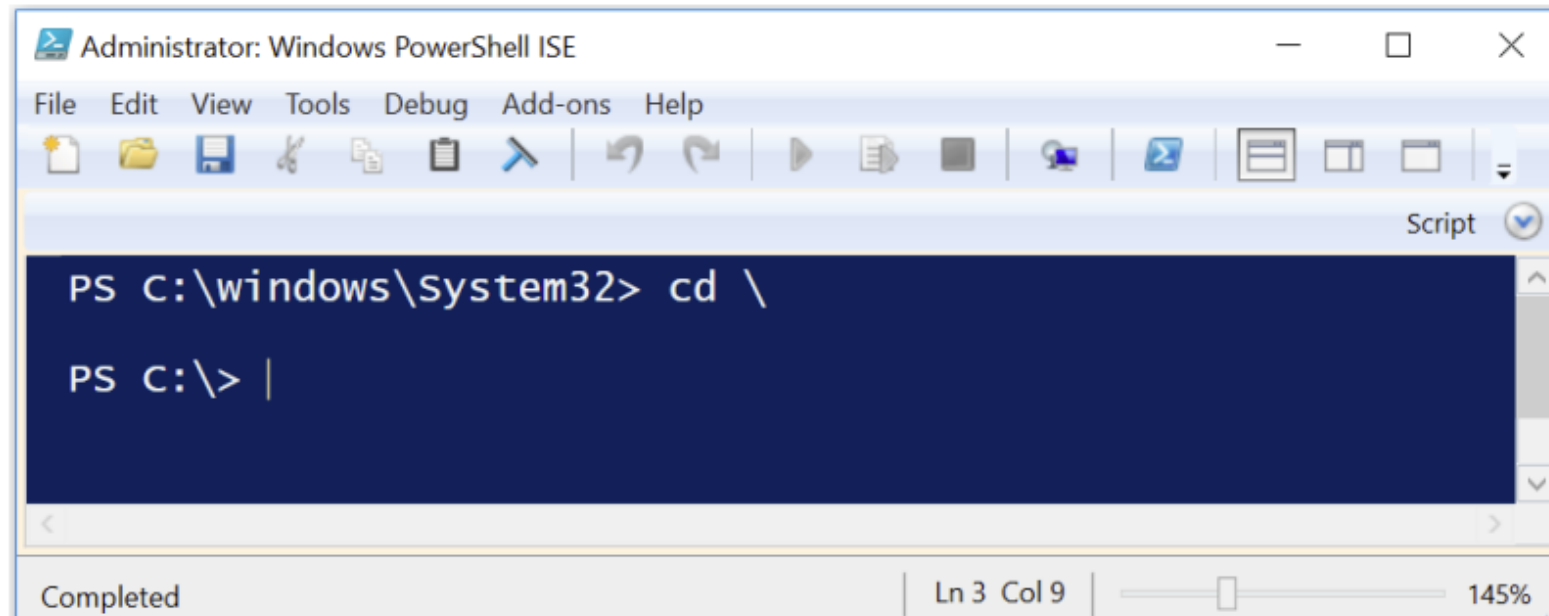
cd system32

כעת, הקלידו

**cd **

לאן עברתם?

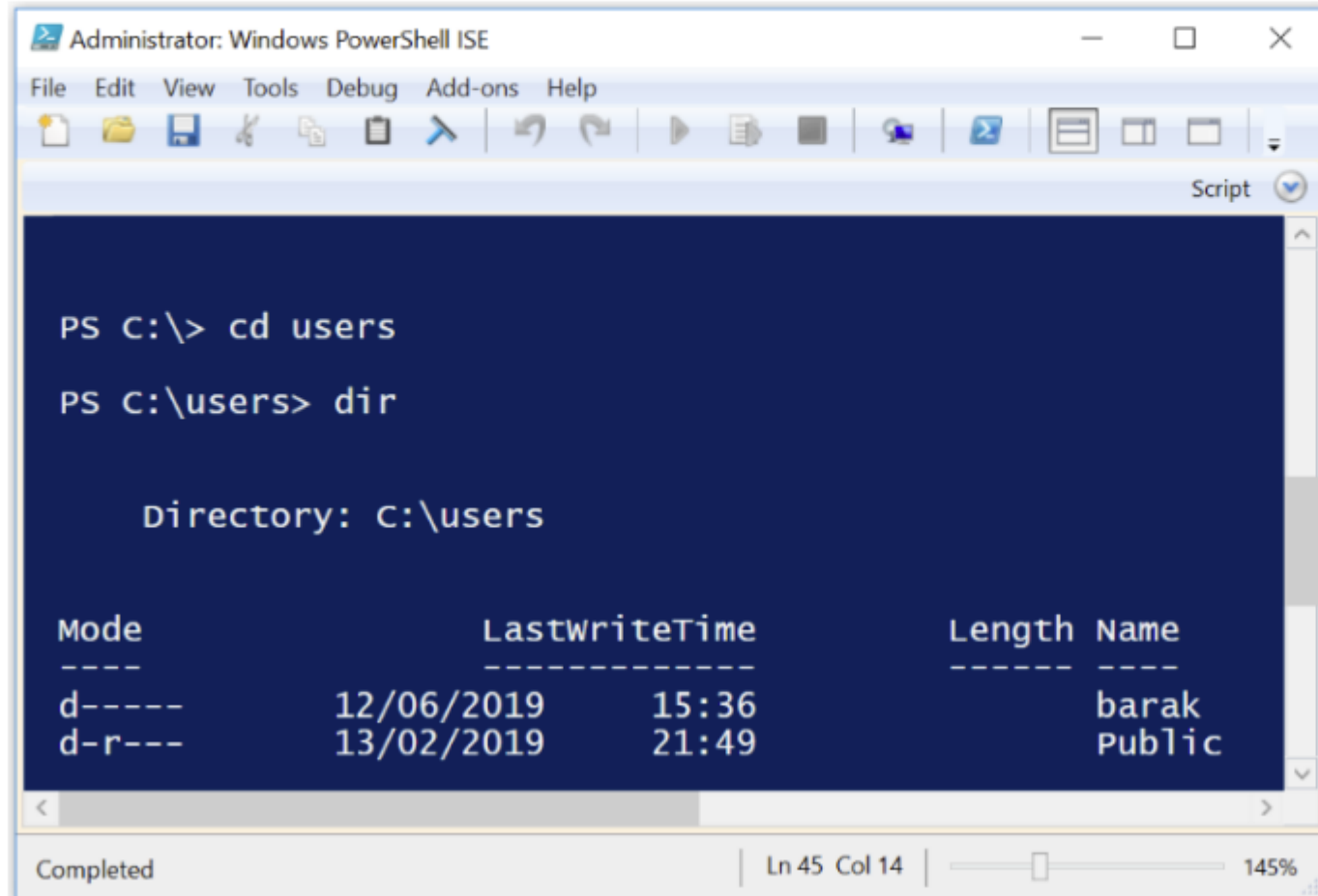
אכן, עברתם הישר אל תיקיית השורש, ה-root, של ההתקן החומרתי בו הקבצים שלכם מאוחסנים.

A screenshot of the Windows PowerShell ISE (Integrated Scripting Environment) window. The title bar reads "Administrator: Windows PowerShell ISE". The menu bar includes "File", "Edit", "View", "Tools", "Debug", "Add-ons", and "Help". Below the menu is a toolbar with various icons for file operations and execution. The main text area has a dark blue background and shows the command prompt "PS C:\windows\System32> cd \" followed by a new line "PS C:\> |". The status bar at the bottom indicates "Completed", "Ln 3 Col 9", and a zoom level of "145%".

```
Administrator: Windows PowerShell ISE
File Edit View Tools Debug Add-ons Help
PS C:\windows\System32> cd \
PS C:\> |
Completed | Ln 3 Col 9 | 145%
```

גשו באמצעות פקודת cd אל התיקיה users.

נשתמש בפקודת dir כדי לצפות בכל הקבצים ותתי התיקיות שקיימים בספריה בה אנחנו נמצאים



The screenshot shows the Windows PowerShell ISE interface. The command prompt shows the user has navigated to the 'users' directory and executed the 'dir' command. The output displays a directory listing for 'C:\users' with columns for Mode, LastWriteTime, Length, and Name. Two entries are visible: 'barak' with mode 'd-----' and 'Public' with mode 'd-r---'.

```
PS C:\> cd users
PS C:\users> dir

Directory: C:\users

Mode                LastWriteTime         Length Name
----                -
d-----          12/06/2019   15:36         barak
d-r---           13/02/2019   21:49        Public
```

במקרה הזה, תחת התיקיה users יש שתי תיקיות. התו "d" מתחת לעמודה "mode" פירושו תיקיה. התו "r" פירושו read only, כלומר התיקיה משמשת לקריאה בלבד. אין אפשרות ליצור קבצים חדשים בתוך התיקיה.

תוכנת Powershell כוללת אוסף של פקודות שמגיעות כחלק מהתוכנה וניתן להריץ אותן פשוט על ידי קריאה בשמן. פקודות אלו נקראות cmdlets, או ביחיד- cmdlet. אלו הן "פקודות קלות משקל" כפי שהן מתוארות על ידי Microsoft.

ובכן, ל-Powershell יש אוסף גדול למדי של פקודות. לכל הפקודות של Powershell יש אותו מבנה: שתי מילים, שמופרדות עם מקף, המילה הראשונה קובעת איזו פעולה תבוצע, לדוגמה:

- Get
- Set
- Test
- Resolve
- New
- Clear

והמילה השנייה קובעת על מה בדיוק תבוצע הפעולה.

ראינו רק כמות קטנה למדי של cmdlets. איך אפשר לראות את רשימת כל ה-cmdlets הנתמכים על ידי Powershell?

פשוט מאד- כיתבו Get-Command.

לחלק מהפקודות הסטנדרטיות יש מה שנקרא Alias, כלומר כינויים. אלו שמות מקוצרים, שכאשר כותבים אותם מקבלים בעצם את אותה פקודה כמו הפקודה בעלת הכינוי הארוך.

ומה לגבי פקודות dir ו-cd שהכרנו? האם גם הן aliases של פקודות אחרות?

בוודאי שכן! נסו בעצמכם את הפקודות הבאות:

- Set-Location c:\Windows
- Get-ChildItem

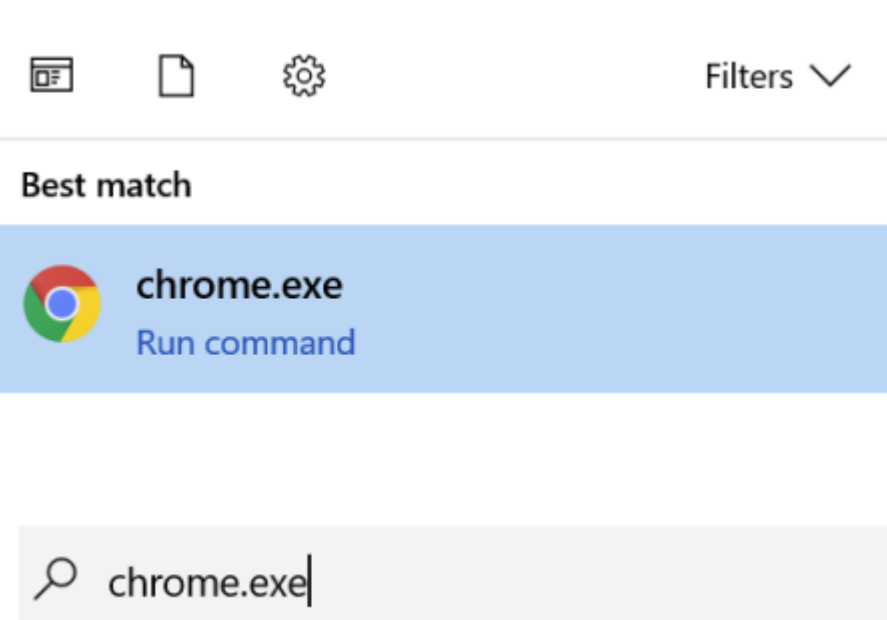
ראינו רק דוגמאות ספורות ל-aliases. איך אפשר לראות את כל ה-aliases של כל ה-cmdlets?

פשוט מאד- כיתבו Get-Alias.

הריצו מ-Powershell את התוכנות הבאות:

- Calculator
- Notepad
- Explorer

מה שמשותף לכל התוכנות הללו הוא שאין צורך לציין היכן הן נמצאות, המחשב כבר יודע למצוא אותן לבד. אולם זה לא תמיד המצב. לדוגמה, בהנחה שדפדפן chrome מותקן אצלכם, לא תצליחו להפעיל אותו באמצעות הפקודה "chrome".



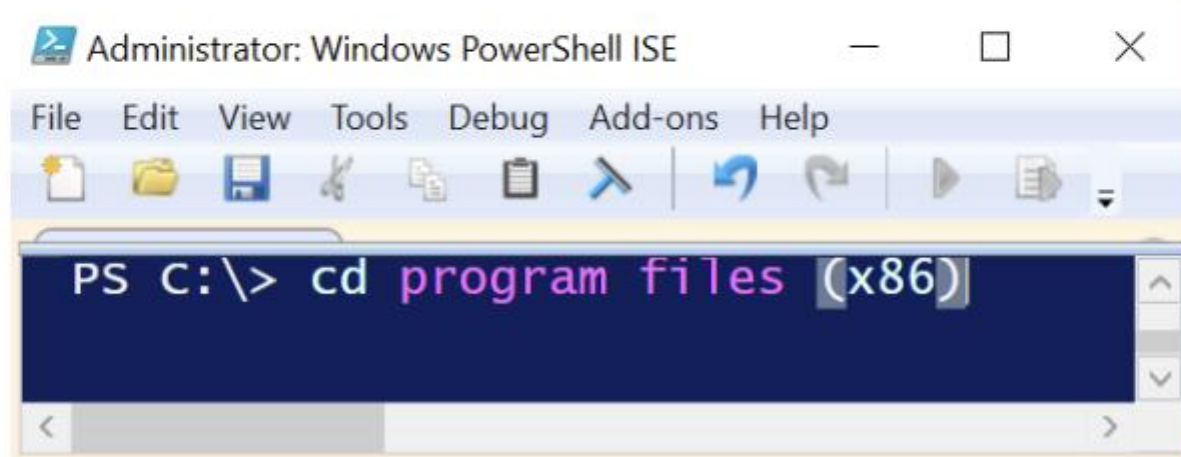
באיזו תיקיה נמצא הקובץ chrome.exe, שמריץ את chrome?
כדי למצוא את התיקיה הנ"ל, נחפש בשורת החיפוש של Windows:

נקליק קליק ימני על תוצאת החיפוש ונבחר באפשרות "הצג בתיקיה", וכך נמצא את הקובץ המבוקש בתיקיה:

This PC > Windows (C:) > Program Files (x86) > Google > Chrome > Application		
☐ Name	Date modified	Type
 75.0.3770.100	18/06/2019 20:05	File folder
 SetupMetrics	23/06/2019 16:48	File folder
☒  chrome.exe	18/06/2019 03:42	Application
 chrome.VisualElementsManifest.xml	18/06/2019 20:05	XML Document
 chrome_proxy.exe	18/06/2019 03:42	Application
 master_preferences	23/08/2018 22:57	File

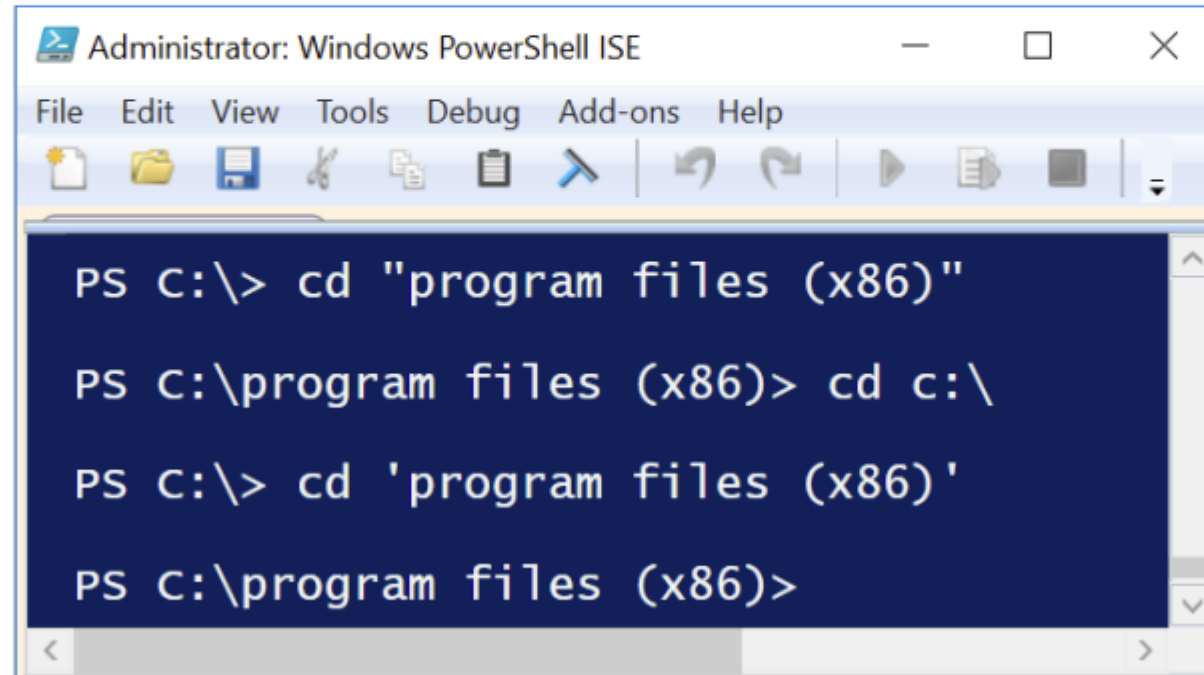
כעת כשאנחנו יודעים היכן נמצא הקובץ chrome.exe, נרצה להריץ אותו. נבצע זאת בשתי דרכים, כל דרך תלמד אותנו משהו שונה על שימוש ב-Powershell.

נשנה את התיקיה בה Powershell נמצא כך שתהיה התיקיה בה נמצא הקובץ chrome.exe. נסו להגיע לתיקיה c:\program files (x86)



```
Administrator: Windows PowerShell ISE
File Edit View Tools Debug Add-ons Help
PS C:\> cd program files (x86)
```

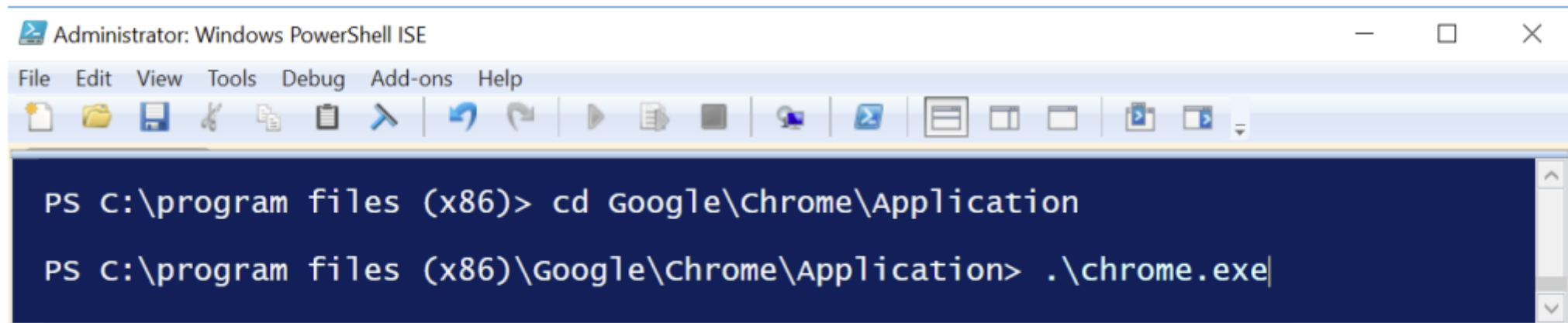
נכון, קבלנו שגיאה... פקודת cd לא יודעת לטפל בשם תיקיה שיש בו רווחים. מה נעשה? נהפוך את שם התיקיה למחרוזת, באמצעות הוספת גרשיים או כפולים, כמו בדוגמאות הבאות:

A screenshot of the Windows PowerShell ISE (Integrated Scripting Environment) window. The title bar reads "Administrator: Windows PowerShell ISE". The menu bar includes "File", "Edit", "View", "Tools", "Debug", "Add-ons", and "Help". Below the menu is a toolbar with icons for file operations and execution. The main text area has a dark blue background and contains the following PowerShell commands and their prompts:

```
PS C:\> cd "program files (x86)"  
PS C:\program files (x86)> cd c:\  
PS C:\> cd 'program files (x86)'  
PS C:\program files (x86)>
```

כעת יש לנו את הכלים להגיע עד התיקיה שבה נמצא הקובץ chrome.exe.

לאחר שהגענו לתיקיה, ננסה להריץ כרום. איך עושים זאת? כתיבה של "chrome" או "chrome.exe" לא יועילו. הסיבה היא ש-Powershell לא מזהה את chrome בתור פקודה, היא אינה חלק מאוסף הפקודות המוכרות לו. כדי לגרום ל-Powershell להריץ את chrome, יש שתי אפשרויות. אפשרות אחת היא להגדיר את הנתוב של chrome בתוך משתנה הסביבה PATH. אפשרות אחרת היא פשוט לתת לו להבין שזוהי תוכנית שעליו להריץ מהמיקום הנוכחי בו אנחנו נמצאים. נעשה זאת באמצעות כתיבת נקודה ולאחריה קו נטוי, כמו בלינוקס, למי שמכיר:

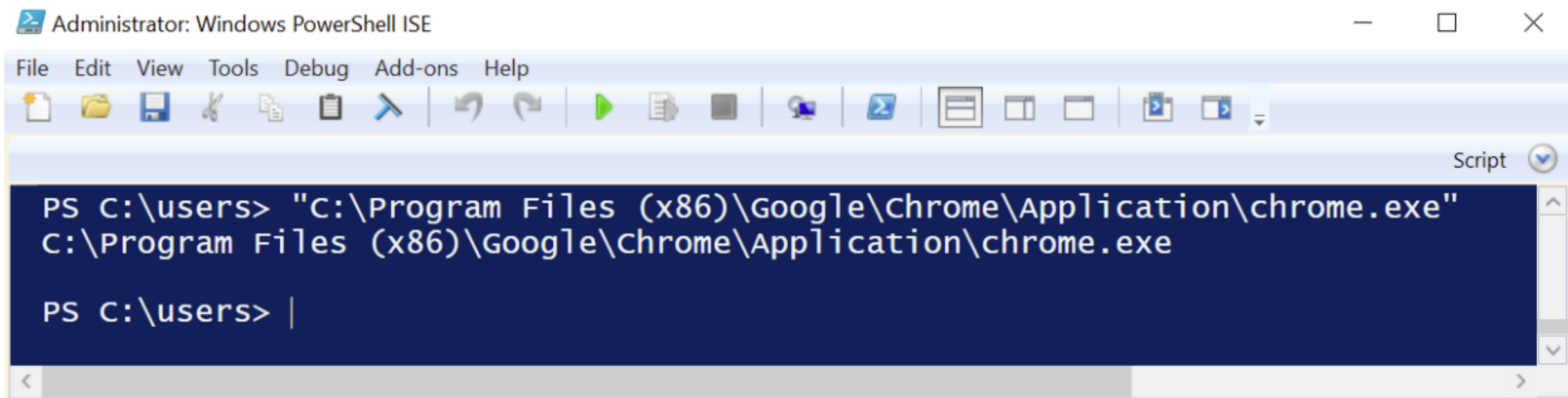
A screenshot of the Windows PowerShell ISE (Integrated Scripting Environment) window. The title bar reads "Administrator: Windows PowerShell ISE". The menu bar includes "File", "Edit", "View", "Tools", "Debug", "Add-ons", and "Help". Below the menu is a toolbar with various icons for file operations and execution. The main console area has a dark blue background and displays two commands entered in white text. The first command is "PS c:\program files (x86)> cd google\Chrome\Application". The second command is "PS c:\program files (x86)\Google\Chrome\Application> .\chrome.exe".

```
Administrator: Windows PowerShell ISE
File Edit View Tools Debug Add-ons Help
PS c:\program files (x86)> cd google\Chrome\Application
PS c:\program files (x86)\Google\Chrome\Application> .\chrome.exe
```

והנה- עובד. ראינו שהצירוף ".\" משמש כדי לומר "המיקום הנוכחי שלנו".

נבצע את אותו דבר בדרך אחרת, וכך נלמד פקודה נוספת.

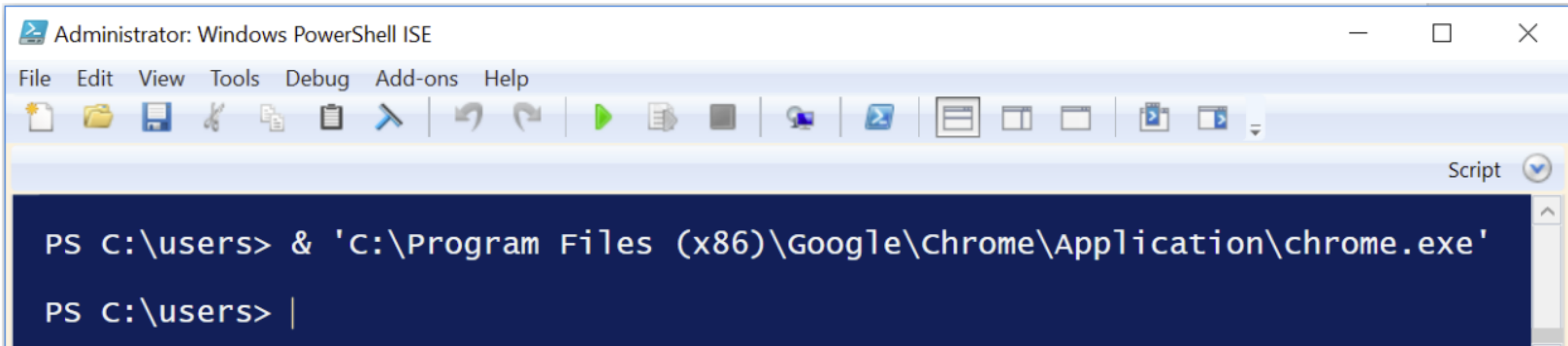
חיזרו לתיקיית C:\ ומשם המשיכו לתיקיית users. אנחנו יודעים כבר מה הנתיב שבו נמצא הקובץ שברצוננו להריץ. האם אנחנו יכולים להריץ אותו בלי לשנות נתיב? ננסה לבצע את הדברים כפי שלמדנו עד כה- ניצור מחרוזת עם גרשיים ובתוכה יהיה הנתיב המלא –

A screenshot of the Windows PowerShell ISE (Integrated Scripting Environment) window. The title bar reads "Administrator: Windows PowerShell ISE". The menu bar includes "File", "Edit", "View", "Tools", "Debug", "Add-ons", and "Help". The toolbar contains various icons for file operations, editing, and execution. The main text area has a dark blue background and shows the following command and output:

```
PS C:\users> "C:\Program Files (x86)\Google\Chrome\Application\chrome.exe"  
C:\Program Files (x86)\Google\Chrome\Application\chrome.exe  
  
PS C:\users> |
```

The command is enclosed in quotes. The output shows the full path to the Chrome executable. The window also features a "Script" button in the top right corner and a scrollbar at the bottom.

לצערנו כל מה שקרה זה שהודפס למסך מה שנמצא בתוך המחרוזת שיצרנו. חסר לנו משהו. מה שחסר הוא הסימן & שמשמעותו "התייחס למה שכתוב במחרוזת הבאה בתור פקודה והרץ אותה". נריץ את אותה פקודה עם הסימן & בתחילתה (שימו לב שהשימוש בגרש יחיד הוא זהה לשימוש בגרשיים כפולים, נחליף ביניהם לעיתים כדי שתוכלו להתרגל לכך):

A screenshot of the Windows PowerShell ISE (Integrated Scripting Environment) window. The title bar reads "Administrator: Windows PowerShell ISE". The menu bar includes "File", "Edit", "View", "Tools", "Debug", "Add-ons", and "Help". The toolbar contains various icons for file operations, editing, and execution. The main text area has a dark blue background and shows the command:

```
PS C:\users> & 'C:\Program Files (x86)\Google\Chrome\Application\chrome.exe'
```

 Below this, the prompt is ready for input:

```
PS C:\users> |
```

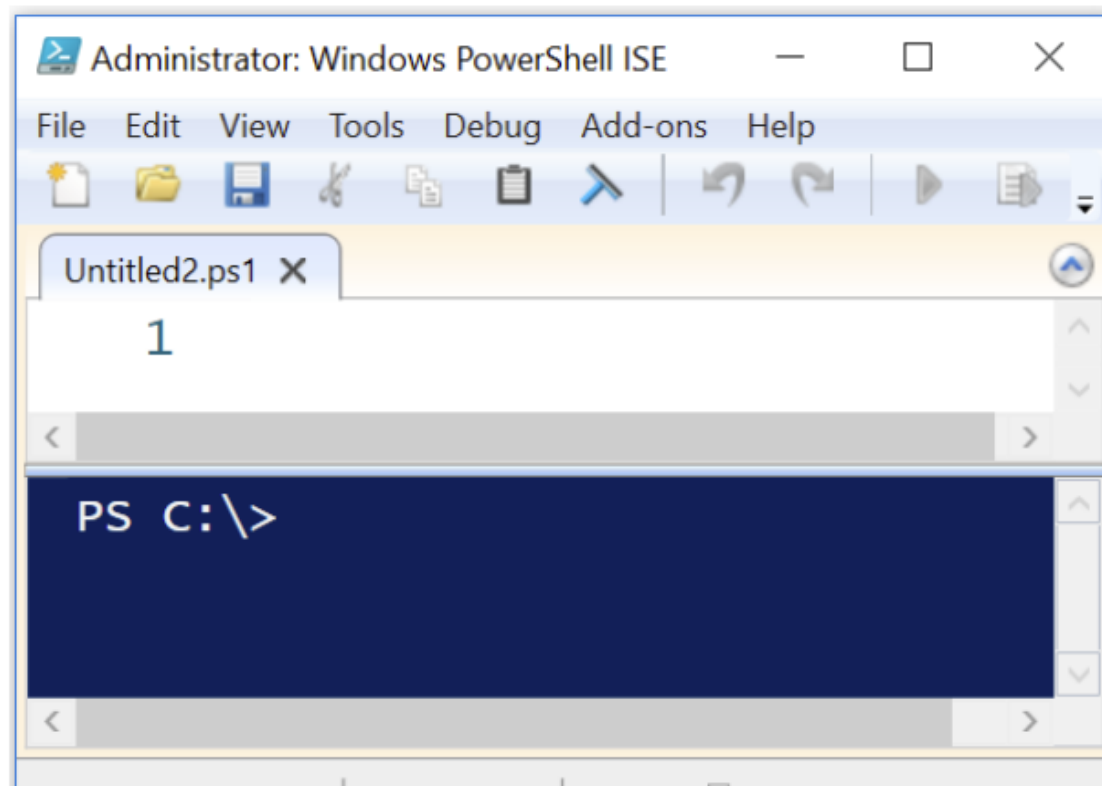
 In the top right corner of the text area, there is a "Script" button with a dropdown arrow.

וזהו זה, כרום עלה ורץ.

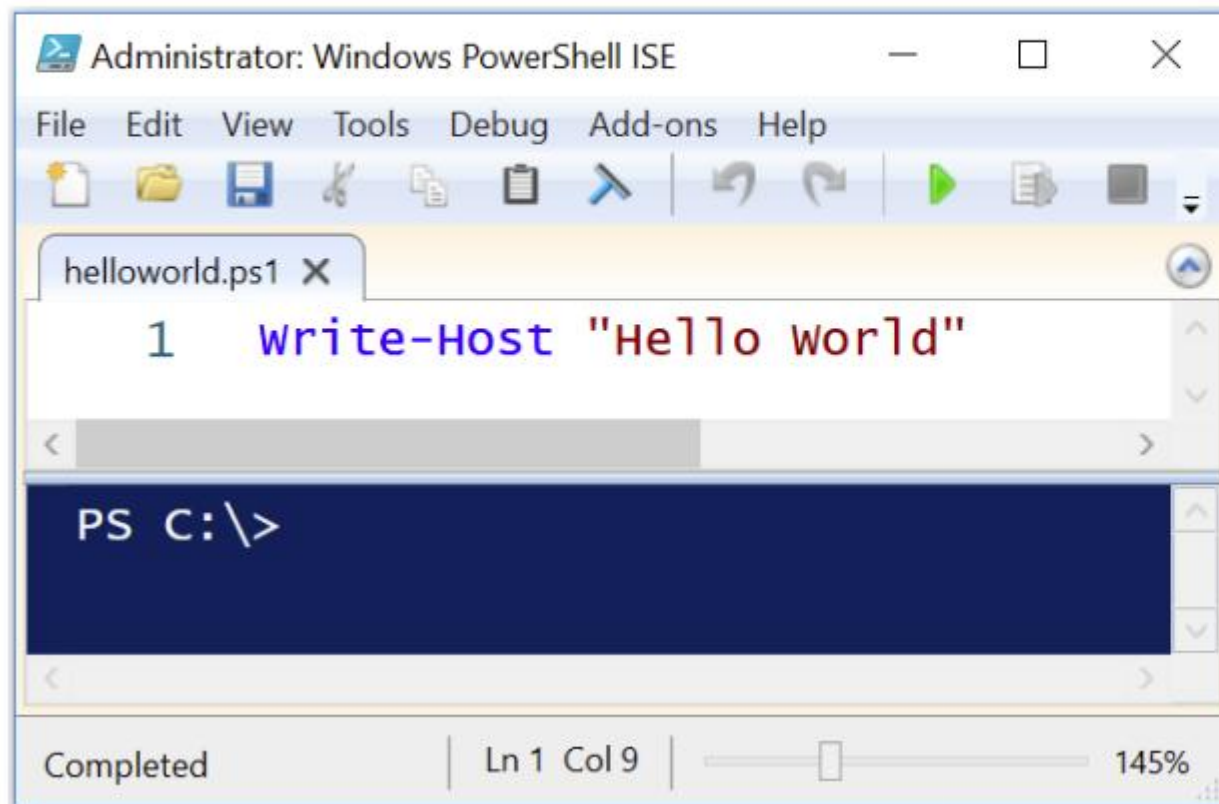
שימו לב שלא הזדקקנו לסימן הנקודה בתחילת המחרוזת, מכיוון שנתנו את הנתיב המלא, ואילו סימן הנקודה אומר שיש להתחיל מהתיקיה הנוכחית בה אנו נמצאים.

האזור הלבן של המסך שלכם משמש לכתיבת סקריפטים. בדומה לסקריפט של פייתון, גם סקריפט של Powershell כולל אוסף פקודות, כאשר הנוחות העקרית בסקריפט היא היכולת להריץ את כולם בבת אחת, פעם אחר פעם בלי לכתוב הכל מחדש, ולדבג בעיות באמצעות breakpoint.

אם מסך הסקריפטים אינו מופיע, לחצו על החץ בצד ימין למעלה.



כיתבו את הקוד הבא ושימרו את הסקריפט בשם כלשהו, לדוגמה helloworld:



The screenshot shows the Windows PowerShell ISE interface. The title bar reads "Administrator: Windows PowerShell ISE". The menu bar includes "File", "Edit", "View", "Tools", "Debug", "Add-ons", and "Help". The toolbar contains icons for file operations (New, Open, Save, Cut, Copy, Paste) and execution (Run, Stop). A single tab titled "helloworld.ps1" is open. The script content is as follows:

```
1 Write-Host "Hello world"
```

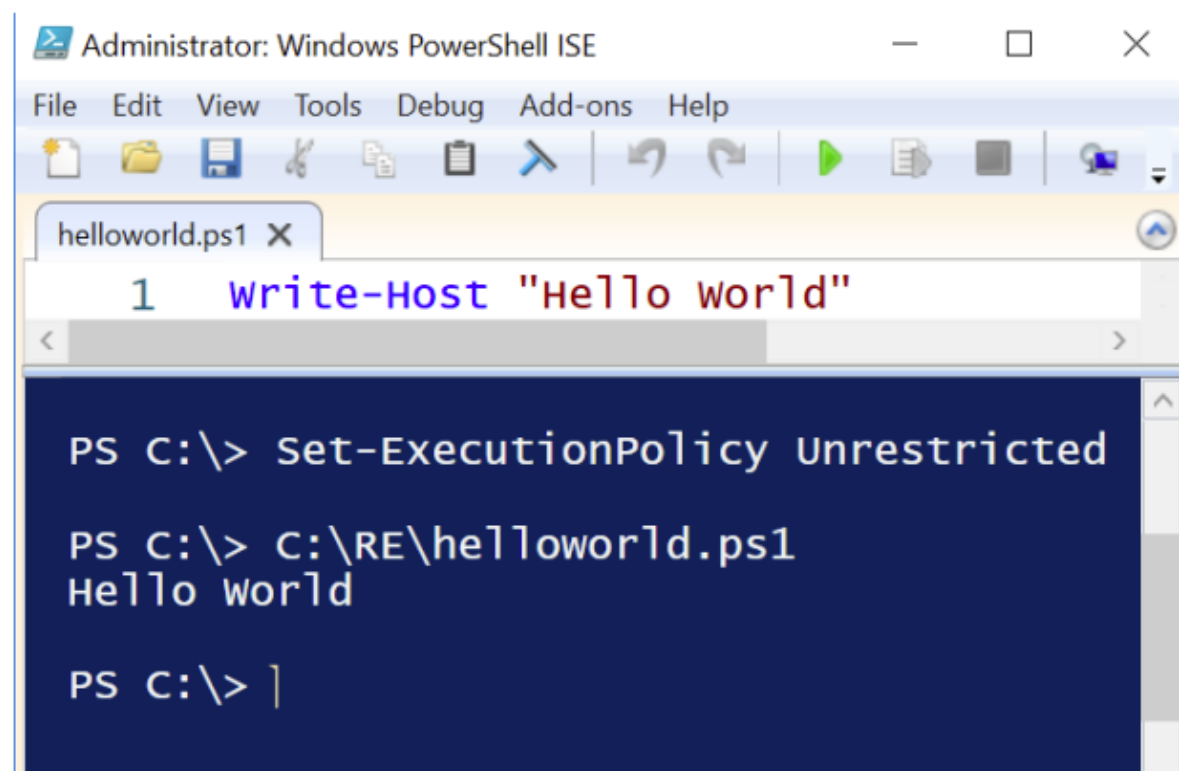
Below the script editor is a dark blue console window with the prompt "PS C:\>". The status bar at the bottom indicates "Completed", "Ln 1 Col 9", and a zoom level of "145%".

כעת לחצו על החץ הירוק למעלה, ו... אופס! הסקריפט לא רץ. בואו נראה מה הודעת השגיאה שאנחנו מקבלים:

```
PS C:\> C:\RE\helloworld.ps1
File C:\RE\helloworld.ps1 cannot be loaded because running scripts is disabled
on this system. For more information, see about_Execution_Policies at
```

הממם. מסתבר שהבעיה היא ש-Powershell מנוע מלהריץ סקריפטים. זה סוג של צעד הגנה מפני תוכנות זדוניות שעלולות להריץ סקריפטים במחשב שלנו, אך כפי שנראה מיד, זהו צעד הגנה לא ממש יעיל. הנה, נבטל את ההגנה.

כיתבו set-ExecutionPolicy ועברו עם הטאב על האפשרויות עד שתגיעו לאפשרות Unrestricted. לחצו enter, אשרו שוב שזה אכן מה שאתם רוצים, וזהו. כעת הריצו את הסקריפט.



The screenshot shows the Windows PowerShell ISE interface. The title bar reads "Administrator: Windows PowerShell ISE". The menu bar includes File, Edit, View, Tools, Debug, Add-ons, and Help. The toolbar contains icons for file operations and execution. A single tab labeled "helloworld.ps1" is open. The script editor shows a single line of code: `1 write-host "Hello world"`. The console window at the bottom displays the following commands and output:

```
PS C:\> Set-ExecutionPolicy Unrestricted

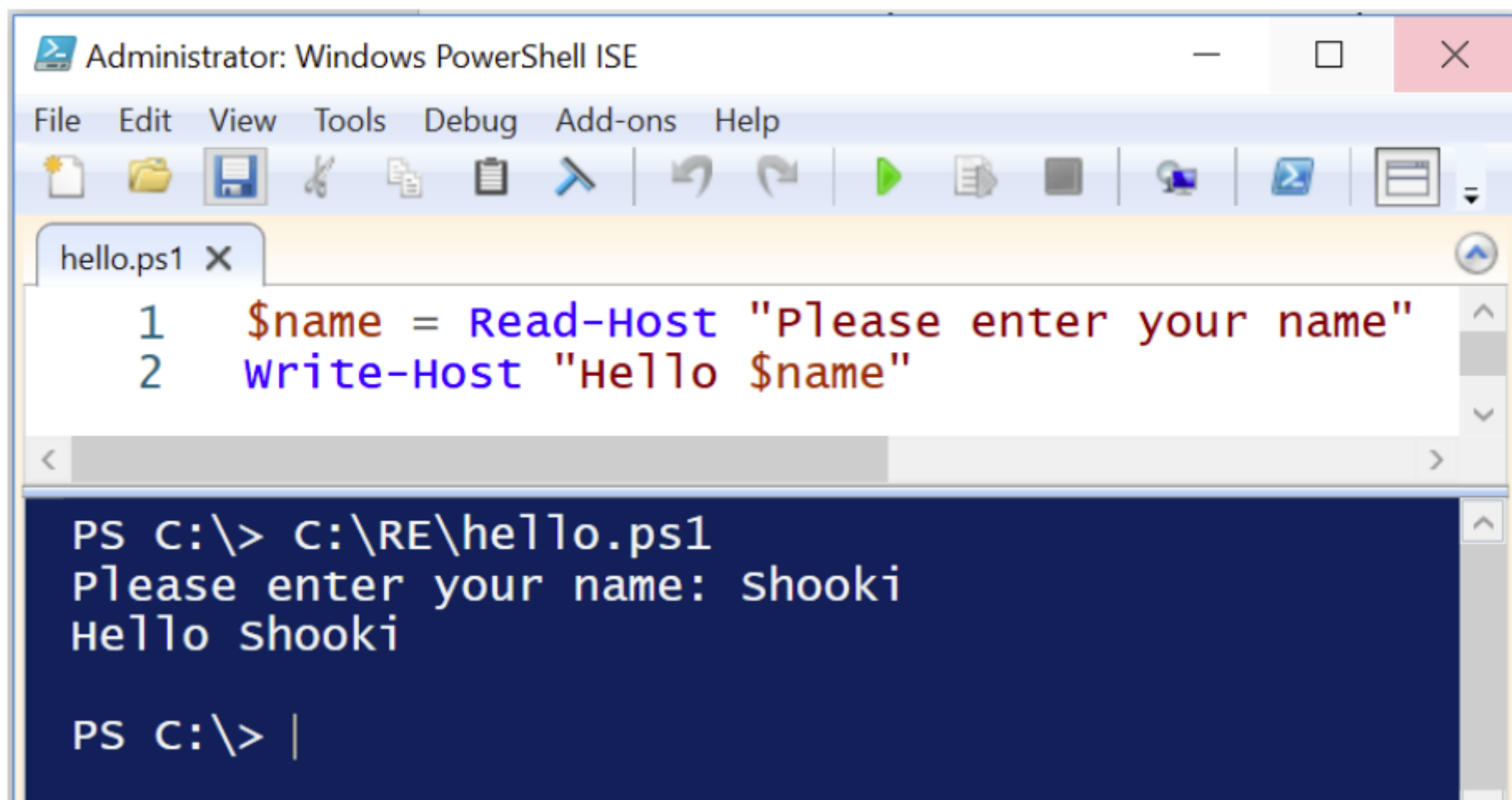
PS C:\> C:\RE\helloworld.ps1
Hello world

PS C:\> |
```

המשמעות מבחינת אבטחה, היא שמי שיש לו גישה למחשב שלכם יכול לבצע כל פעולה שהוא רוצה, אין שום מדיניות אבטחה שמונעת ממנו לעשות הכל, ואם יש כזו- ניתן לעקוף אותה. זהו שיעור שכדאי לזכור.

קבלת קלט והגדרת משתנים

ניצור כעת סקריפט חדש, שיודע לברך אותנו לשלום באופן אישי. הסקריפט יצטרך לקלוט את השם שלנו ולכתוב לנו הודעה אישית.



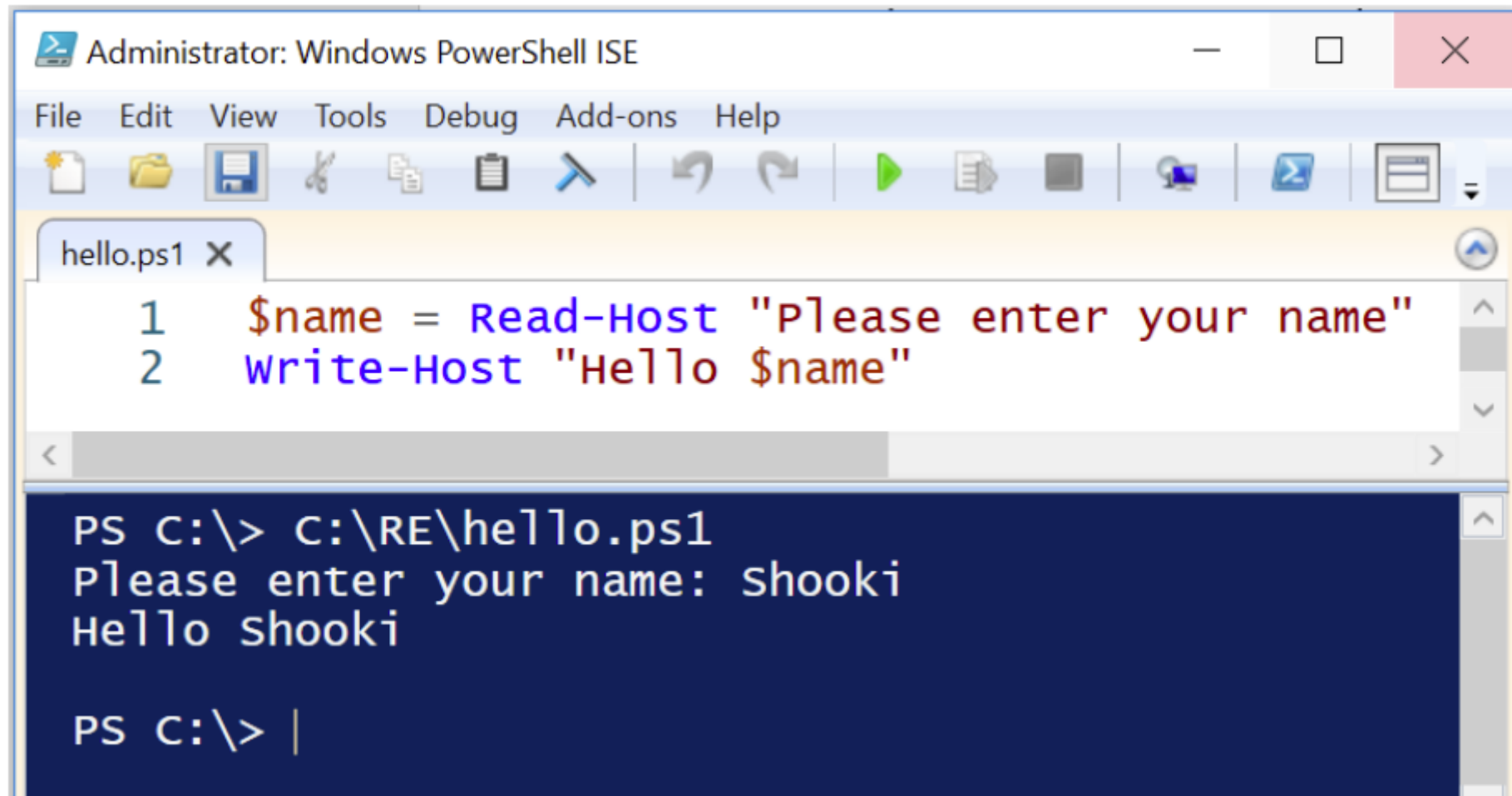
The screenshot shows the Windows PowerShell ISE interface. The title bar reads "Administrator: Windows PowerShell ISE". The menu bar includes File, Edit, View, Tools, Debug, Add-ons, and Help. The toolbar contains icons for file operations and execution. A tab labeled "hello.ps1" is open. The script content is as follows:

```
1 $name = Read-Host "Please enter your name"
2 Write-Host "Hello $name"
```

The console output at the bottom shows the script being executed from the C:\ directory, followed by the prompt "Please enter your name:" and the user input "Shooki", resulting in the output "Hello Shooki".

```
PS C:\> C:\RE\hello.ps1
Please enter your name: Shooki
Hello Shooki

PS C:\> |
```

A screenshot of the Windows PowerShell ISE (Integrated Scripting Environment) window. The title bar reads "Administrator: Windows PowerShell ISE". The menu bar includes "File", "Edit", "View", "Tools", "Debug", "Add-ons", and "Help". Below the menu is a toolbar with various icons for file operations, editing, and execution. A tab labeled "hello.ps1" is open. The script editor contains two lines of PowerShell code:

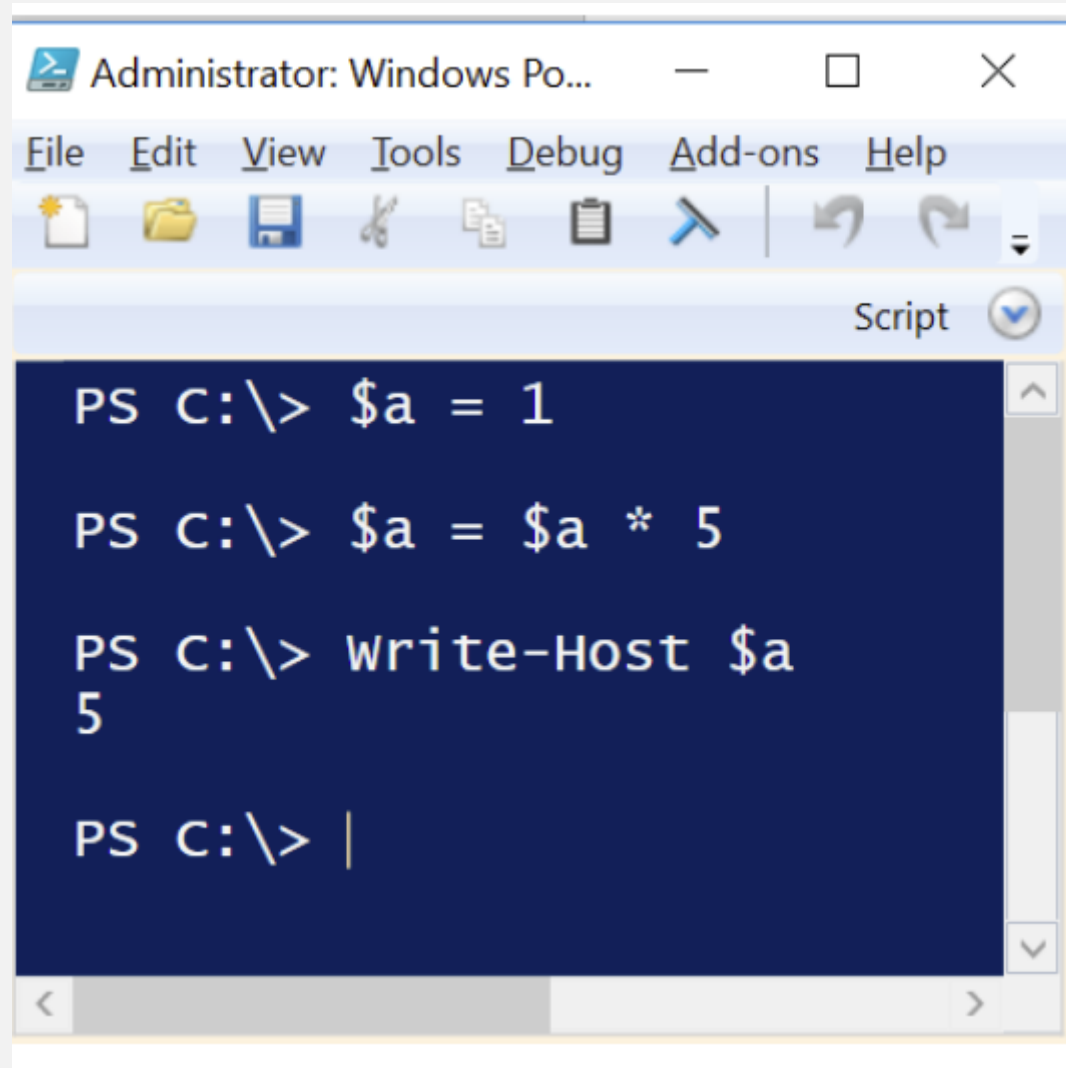
```
1 $name = Read-Host "Please enter your name"
2 Write-Host "Hello $name"
```

The console window at the bottom shows the execution of the script. It displays the prompt "PS C:\> C:\RE\hello.ps1", followed by the prompt "Please enter your name:" and the user input "Shooki". The output is "Hello Shooki". The console prompt is now "PS C:\> |".

הפקודה Read-Host משמשת, כפי שאפשר לנחש מיד, לקריאת קלט ממסך המשתמש.

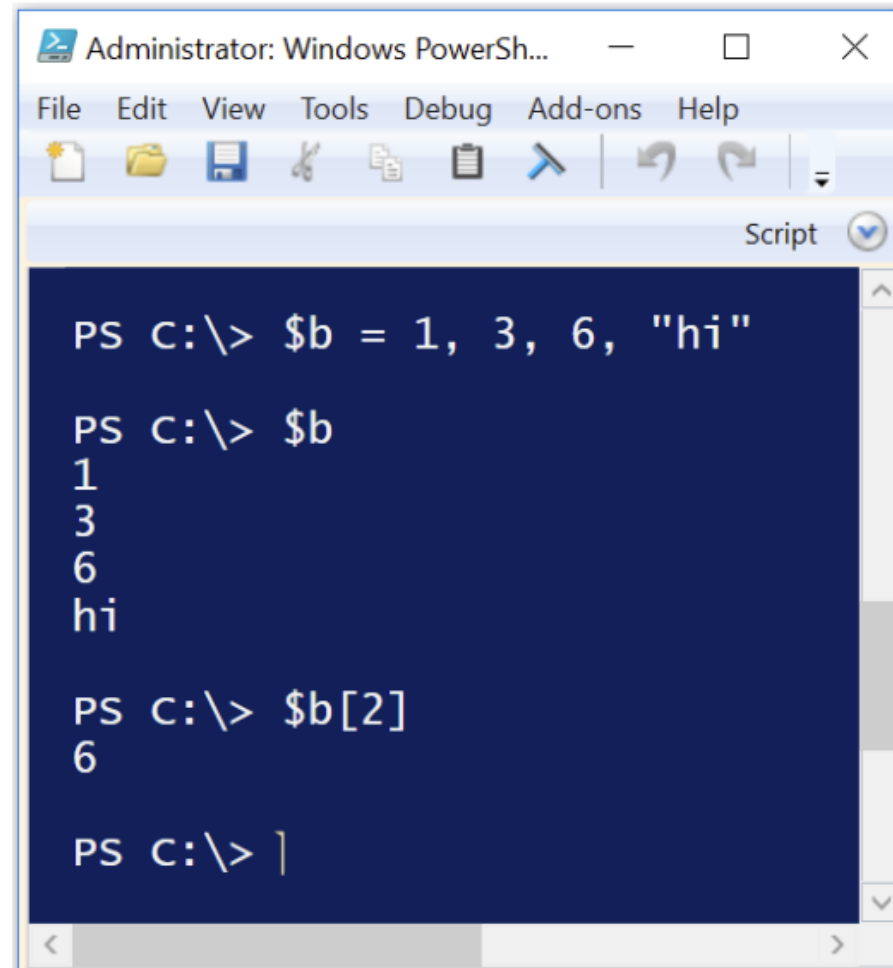
שימו לב לדבר חדש שאנחנו נתקלים בו- משתנה. הגדרנו משתנה בשם name, ששומר את הקלט של המשתמש. סימן ה-\$ לפני name אומר ל-Powershell שמדובר במשתנה, ולא – נניח – בפונקציה שעליו להכיר. כל פעם שנרצה להתייחס למשתנה, נצטרך להוסיף לפניו את סימן ה-\$.

הגדירו משתנה בשם a , שערכו שווה ל-1. כיפלו את a ב-5. הדפיסו למסך את a .



```
Administrator: Windows Po...
File Edit View Tools Debug Add-ons Help
Script
PS C:\> $a = 1
PS C:\> $a = $a * 5
PS C:\> write-Host $a
5
PS C:\> |
```

ניתן להגדיר משתנה שהוא רשימה של ערכים. האיברים ברשימה לא חייבים להיות מאותו טיפוס ואפשר לערבב בין טיפוסים שונים. הגדרת רשימה מתבצעת פשוט על ידי הוספת פסיקים בין הערכים השונים. ניתן לגשת לכל איבר ברשימה באמצעות סוגריים מרובעים והאינדקס של האיבר ברשימה

A screenshot of a Windows PowerShell console window titled "Administrator: Windows PowerShell". The window has a menu bar with "File", "Edit", "View", "Tools", "Debug", "Add-ons", and "Help". Below the menu is a toolbar with icons for file operations and navigation. The main area has a dark blue background with white text. It shows three commands and their outputs: 1. Command: `$b = 1, 3, 6, "hi"`. Output: `1`, `3`, `6`, `hi`. 2. Command: `$b`. Output: `1`, `3`, `6`, `hi`. 3. Command: `$b[2]`. Output: `6`. The prompt `PS C:\>` is visible at the start of each line. The window has standard Windows window controls (minimize, maximize, close) in the top right corner.

```
Administrator: Windows PowerShell
File Edit View Tools Debug Add-ons Help
Script
PS C:\> $b = 1, 3, 6, "hi"

PS C:\> $b
1
3
6
hi

PS C:\> $b[2]
6

PS C:\> |
```

נניח שאנחנו רוצים לבצע פעולה מסויימת, אבל אנחנו לא מכירים את הפעולה הספציפית שעושה את מה שאנחנו מבקשים. לדוגמה, יש לנו את תוכנת Notepad במחשב (היא לא פתוחה אצלכם? פיתחו אותה!) ואנחנו רוצים לסגור אותה רק באמצעות Powershell.

ב-Powershell יש מנגנון עזרה נחמד, שמאפשר לנו למצוא את כל המידע על מה שאנחנו צריכים וכולל דוגמאות. הדבר הראשון שנצטרך לעשות הוא לעדכן אותו. כיתבו Update-Help וכל המידע יירד מהאינטרנט אל המחשב שלכם.

אוקי, אנחנו רוצים לחפש משהו שקשור לסגירת תוכנות במחשב. כל תוכנה שרצה במחשב היא Process. נשתמש בפקודה Get-Help, שה-alias שלה הוא פשוט help, או man כמו בלינוקס.

נכתוב Get-Help process ונקבל את רשימת כל הדברים ב-Powershell שיש להם קשר למילה Process.

```
PS C:\> get-help process
```

Name	Category	Module	Synopsis
----	-----	-----	-----
Enter-PSHostProcess	Cmdlet	Microsoft.PowerShell.Core	Connects to and enters i
Exit-PSHostProcess	Cmdlet	Microsoft.PowerShell.Core	Closes an interactive se
Get-PSHostProcessInfo	Cmdlet	Microsoft.PowerShell.Core	
Debug-Process	Cmdlet	Microsoft.PowerShell.M...	Debugs one or more proces
Get-Process	Cmdlet	Microsoft.PowerShell.M...	Gets the processes that
Start-Process	Cmdlet	Microsoft.PowerShell.M...	Starts one or more proces
Stop-Process	Cmdlet	Microsoft.PowerShell.M...	Stops one or more running
Wait-Process	Cmdlet	Microsoft.PowerShell.M...	waits for the processes
ConvertTo-ProcessMitigationPolicy	Cmdlet	ProcessMitigations	ConvertTo-ProcessMitigat
Get-ProcessMitigation	Cmdlet	ProcessMitigations	Get-ProcessMitigation...
Set-ProcessMitigation	Cmdlet	ProcessMitigations	Set-ProcessMitigation...

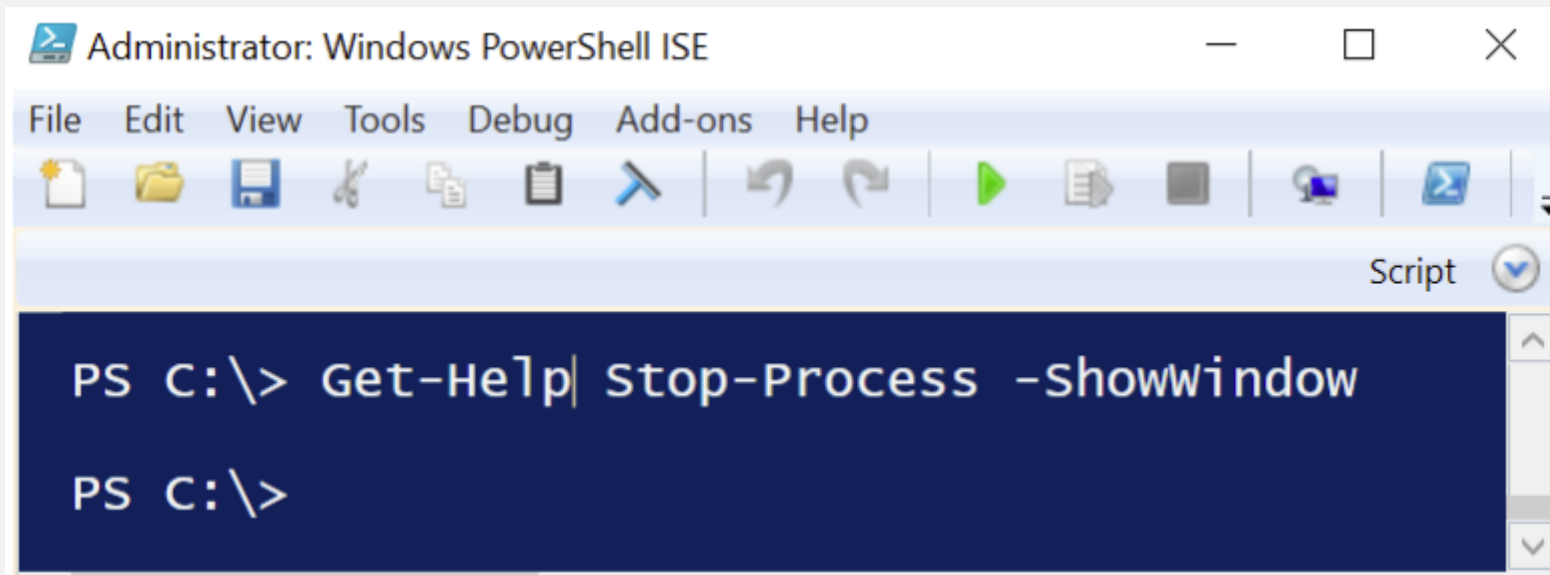
אנחנו רואים שישנן כ-10 תוצאות שקשורות למילה Process, כל התוצאות הן cmdlets. נתבונן בשמות של

הפקודות ובתיאורים שלהם. האם אתם יכולים למצוא משהו שסוגר Process?

נראה ש- Stop-Process היא הפקודה הנכונה. אבל איך משתמשים בה?

אין בעיה, למה לא נבצע `Get-Help Stop-Process`?

טריק נחמד הוא לכתוב את פקודת העזרה כך, עם `ShowWindow` בסוף.

A screenshot of the Windows PowerShell ISE (Integrated Scripting Environment) window. The title bar reads "Administrator: Windows PowerShell ISE". The menu bar includes "File", "Edit", "View", "Tools", "Debug", "Add-ons", and "Help". The toolbar contains various icons for file operations, editing, and execution. The main text area has a dark blue background and shows the command `PS C:\> Get-Help Stop-Process -Showwindow` being entered. Below it, the prompt `PS C:\>` is visible. A vertical scrollbar is on the right side of the text area.

```
Administrator: Windows PowerShell ISE
File Edit View Tools Debug Add-ons Help
PS C:\> Get-Help Stop-Process -Showwindow
PS C:\>
```

יפתח חלון חדש ובו כל הפרטים שאנחנו צריכים:

 Stop-Process Help

Previous

Next

Settings

Synopsis

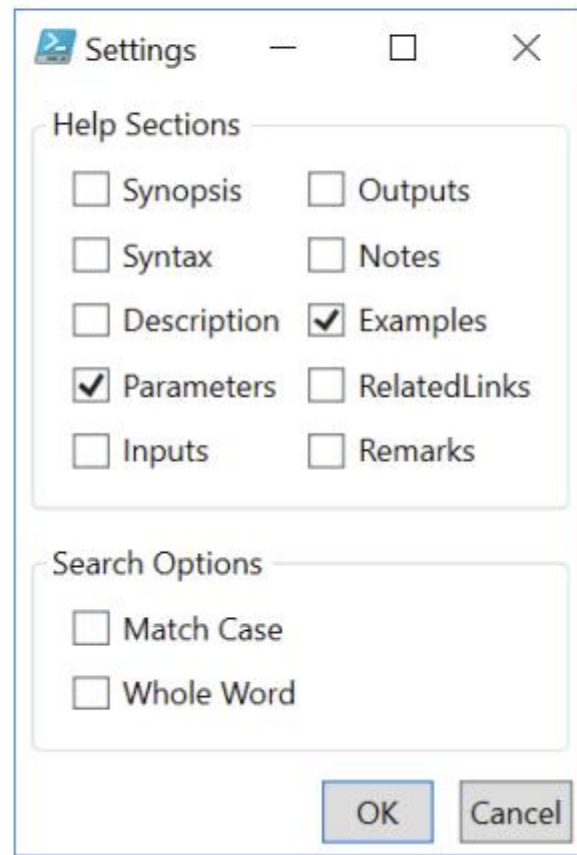
Stops one or more running processes.

Description

The Stop-Process cmdlet stops one or more running processes. You can specify a process by process name or process ID (PID), or pass a process object to Stop-Process . Stop-Process works only on processes running on the local computer.

On Windows Vista and later versions of the Windows operating system, to stop a process that is not owned by the current user, you must start Windows PowerShell by using the Run as administrator option. Also, you are prompted for confirmation unless you specify the Force parameter.

בחלון הזה יש המון מידע. נוכל לצמצם אותו קצת באמצעות ה-settings. הקליקו על הכפתור וסמנו ב-V רק את הפרמטרים והדוגמאות-



כעת נחפש משהו שמגדיר לפקודת Stop-Process מה השם של ה-Process שאותו אנחנו רוצים לסגור.
נחפש "Name" ונמצא כי יש פרמטר כזה-

-Name <String[]>

Specifies the process names of the processes to stop. You can type multiple process names, separated by commas, or use wildcard characters.

עכשיו אנחנו יודעים איך צריכים להראות הפקודה שלנו. היא צריכה להתחיל ב-Stop-Process, לאחר מכן מקף, לאחר מכן Name ובסוף שם התוכנה.

נסו זאת! הצלחתם לסגור את notepad?

אם לא- נסו זאת כך:

```
PS C:\> Stop-Process -name notepad
```

נהדר. ראינו שאנחנו יכולים בעזרת Get-Help לקבל את כל המידע שאנחנו רוצים על כל פקודה שקיימת ב-Powershell. נוכל להמשיך לדוגמה נוספת.

הפעם נרצה לקרוא את כל המידע שיש בקובץ טקסט מסויים ולהדפיס אותו למסך. מאיפה נתחיל?

נכתוב Get-Help file ונקבל את רשימת כל הדברים הקיימים ב-Powershell שיש להם קשר למילה File.

```
PS C:\WINDOWS\system32> get-help file
```

Name	Category	Module	Synopsis
----	-----	-----	-----
Get-FileHash	Function	Microsoft.PowerShell.U...	Computes the hash value f
Import-PowerShellDataFile	Function	Microsoft.PowerShell.U...	
New-TemporaryFile	Function	Microsoft.PowerShell.U...	Creates a temporary file.

לא להבהל מכמות המידע שנחתה עלינו. אנחנו רק צריכים לסנן את מה שאנחנו מחפשים. מצד ימין יש תיאור קצר של הנושא. נחפש תיאור שקשור למה שאנחנו מחפשים- קריאה מקובץ.

New-StorageFileServer	Function	Storage	...
FileSystem	Provider	Microsoft.PowerShell.Core	Provides access to files and directories.
about_Profiles	HelpFile		Describes how to create and use a windows

מסתבר שדבר מה שקוראים לו FileSystem הוא תחת הקטגוריה "Provider". מהו Provider? בהמשך נראה. כרגע חשוב לדעת רק שזה מאפשר גישה לקבצים ותיקיות. בינגו!

נבצע Get-Help נוסף, הפעם על FileSystem.

שוב קבלנו כמות מכובדת מאד של מידע. בפתיחה יש תיאור כללי על מה הדברים ש-FileSystem מאפשר לעשות. מהקריאה מתברר שאפשר לפתוח קובץ, לקרוא, למחוק או לשנות גם קבצים וגם ספריות. לכל פעולה שנרצה יש פקודה מתאימה.

הקובץ בנוי כך שיש אוסף של משימות (Tasks) כאשר לכל משימה יש כותרת ובהמשך מוסבר מה צריך לעשות כדי לבצע אותה. המשימה הראשונה היא פיצול של קובץ לחלקים. כדי לפצל קובץ צריך קודם כל לקרוא את הקובץ... לכן נקרא את התיאור שבהמשך, ומיד ניתקל בפקודה Get-Content. נפלא. לפי הדוגמה זה בדיוק מה שאנחנו צריכים.

טיפ חשוב: Powershell כולל קבצי הדרכה על כל הנושאים החשובים, אפשר למצוא את המדריכים הללו בקלות כיוון שהם מתחילים ב- "about". אם נבצע help about נוכל לראות את רשימת כל קבצי ההדרכה. כדי לצפות במידע שבקובץ ספציפי, צריך רק לכתוב help ואז את הנושא המבוקש, לדוגמה

help about_variables

```
PS C:\> help about
```

Name	Category	Module	Synopsis
----	-----	-----	-----
about_ActivityCommonParameters	HelpFile		Descri...
about_Aliases	HelpFile		Descri...
about_Arithmetic_Operators	HelpFile		Descri...
about_Arrays	HelpFile		Descri...
about_Assignment_Operators	HelpFile		Descri...
about_Automatic_Variables	HelpFile		Descri...
about_Break	HelpFile		Descri...
about_Checkpoint-workflow	HelpFile		Descri...
about_CimSession	HelpFile		Descri...
about_Classes	HelpFile		Descri...

התוצאות הראשונות של הרצת help about

כאשר ביצענו `Get-Help Stop-Process` ראינו שהפקודה `Stop-Process` מקבלת פרמטרים, אחד מהם היה `Name`. כמו שאפשר לנחש, לפונקציה יש המון אפשרויות לקבל פרמטרים. כעת נעבור בצורה מסודרת על איך אפשר לדעת אילו פרמטרים פונקציה מקבלת ואיך משתמשים בהם.

נתחיל ממה שאנחנו כבר יודעים- נבצע `help`:

```
PS C:\> help Get-Process -Showwindow
```

נאפשר את החלק של Syntax, היכן שהוא מוגדר ב-Settings של החלון שנוצר, ונקבל את הפירוט הבא:

Syntax

```
Get-Process [[-Name] <String[]>] [-ComputerName <String[]>] [-FileVersionInfo ] [-Module ] [<CommonParameters>]

Get-Process [-ComputerName <String[]>] [-FileVersionInfo ] -Id <Int32[]> [-Module ] [<CommonParameters>]

Get-Process [-ComputerName <String[]>] [-FileVersionInfo ] -InputObject <Process[]> [-Module ] [<CommonParameters>]

Get-Process -Id <Int32[]> -IncludeUserName  [<CommonParameters>]

Get-Process [[-Name] <String[]>] -IncludeUserName  [<CommonParameters>]

Get-Process -IncludeUserName -InputObject <Process[]> [<CommonParameters>]
```

מה רואים כאן? ראשית, ישנן 6 דרכים להריץ את Get-Process. כל דרך נבדלת מהאחרות בפרמטרים שהיא מקבלת (וכך בעצם Powershell יודע להבחין באיזה אופן המשתמש ביקש להריץ את הפקודה).

נסקור את הדרך הראשונה. בשלב זה נתמקד רק בקטע הבא:

```
[[-Name] <String[]>]
```

הפרמטר הראשון נקרא "Name". אנחנו יודעים שזהו פרמטר כי יש לפניו מקף. לאחר מכן, בתוך סוגריים משולשים, כתוב סוג המשתנה. במקרה הזה הסוג הוא String, הגיוני ששם ה-process יהיה מטיפוס string. עד כאן, אנחנו מבינים כי הפקודה Get-Process מצפה לקבל פרמטר Name מטיפוס מחרוזת. אך ישנם בקטע זה לא פחות מ-3 זוגות של סוגריים מרובעים. נסביר מה תפקידם.

א. הסוגריים המרובעים החיצוניים - `[Name-]<String[]>`. סוגריים אלו משמעותם שלא חובה להשתמש בפרמטר הזה. כלומר אנחנו יכולים לבצע `Get-Process` גם בלי לציין את הפרמטר `name` וגם בלי לתת מחרוזת כלשהי. נסו זאת- כיתבו `Get-Process` ותראו איך הפקודה עובדת, בלי פרמטרים כלל.

ב. הסוגריים המרובעים מסביב ל-`[<String[]> -Name]` - `Name`. סוגריים אלו מציינים שאפשר לא לטרוח לכתוב את הפרמטר `Name-`, ו-`Powershell` יבין לבד כי המחרוזת הראשונה מתייחסת לפרמטר זה. נסו זאת- כיתבו `Get-Process Powershell_ise` וצפו בתוצאה (הכי פשוט לכתוב את ההתחלה של שם ה-`process`, לדוגמה `pow`, ואז ללחוץ על טאב ולתת להשלמה האוטומטית לעבוד).

ג. הסוגריים המרובעים אחרי `<String[]>` `[-Name]` - `String`. סוגריים אלו מציינים שאפשר לתת מערך של ערכים, לא רק אחד. נסו זאת- פיתחו תוכנת notepad, כעת כיתבו את הפקודה הבאה וצפו בתוצאה:

```
PS C:\> Get-Process powershell_ise, notepad
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
-----	-----	-----	-----	-----	--	--	-----
238	14	3100	13296	0.03	29904	1	notepad
950	76	392352	197340	148.55	15940	1	powershell_ise

נמשיך אל הקטע הבא: `[-ComputerName <String[]>]`

יש לנו כבר את כל הידע שצריך בשביל להבין את הפרמטר הזה. נסו לענות בעצמכם על השאלות הבאות:

1. האם חייבים להשתמש בפרמטר הזה?

2. אם החלטנו להשתמש בפרמטר הזה, האם חייבים לכתוב ComputerName או שאפשר לוותר על אזכור שם הפרמטר?

3. מהו הטיפוס שמקבל הפרמטר הזה?

4. האם אפשר לתת לפרמטר הזה מספר אפשרויות?

1. מסביב לפרמטר יש סוגריים מרובעים ולכן לא, לא חייבים להשתמש בו

2. אין סוגריים מרובעים מסביב ל-ComputerName, כלומר אם רוצים להשתמש בו חייבים לציין זאת

3. מחרוזת כמובן

4. בהחלט כן. לאחר סוג הפרמטר יש סוגריים מרובעות [], כלומר הוא מצפה לקבל מספר ערכים

כפי שראינו, יש ל-Powershell לא פחות מ-6 אפשרויות לשימוש בפקודה Get-Process. אם כך, איך Powershell יודע באיזו אפשרות בחרנו להשתמש?

התשובה היא שהפרמטרים מאפשרים זאת. לדוגמה, אם הפקודה שלנו מתחילה כך-

Get-Process -ID

אז בוודאות אנחנו עובדים עם צורת הכתיבה הרביעית, ו-Powershell יצפה שהמשך הכתיבה יהיה לפי הצורה הזו.

פיתחו תוכנת notepad. כעת הציגו את הפרטים של ה-process של notepad. כעת סיגרו את notepad.
רק באמצעות Powershell.

```
PS C:\> notepad
```

```
PS C:\> Get-Process -Name notepad
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
-----	-----	-----	-----	-----	--	--	-----
238	14	3092	13240	0.06	20328	1	notepad

```
PS C:\> Stop-Process -Name notepad
```

בחלק קודם נתקלנו במושג Provider. ראינו שישנו Provider שמספק לנו פונקציות שמאפשרות גישה למערכת הקבצים ולביצוע פעולות שונות עליהן.

כפי שאפשר להניח, המושג Provider הוא מושג כללי יותר. Microsoft לוקחת אוסף של מקורות מידע ומנגישה אותם לתוך סביבת העבודה של Powershell כאילו היו מערכות קבצים. זה אומר שכל מה שאפשר לעשות על קבצים ותיקיות, אפשר לעשות על מקורות מידע נוספים. אז מהם אותם מקורות מידע מסתוריים?

כדי לענות על השאלה, נעשה פשוט מאד help provider. ונחפש תיאורים רלבנטיים. התוצאה הראשונה היא cmdlet שנקרא Get-PSProvider. האותיות PS הן כפי ששמנו לב קיצור של Powershell, לכן יהיה מעניין להריץ את הפונקציה ולראות מה היא מבצעת.

```
PS C:\> Get-PSProvider
```

Name	Capabilities	Drives
----	-----	-----
Registry	ShouldProcess, Transactions	{HKLM, HKCU}
Alias	ShouldProcess	{Alias}
Environment	ShouldProcess	{Env}
FileSystem	Filter, ShouldProcess, Credentials	{C, D}
Function	ShouldProcess	{Function}
Variable	ShouldProcess	{Variable}
Certificate	ShouldProcess	{Cert}
WSMan	Credentials	{WSMan}

קבלנו רשימה של כל ה-Providers שיש בתוך סביבת ה-Powershell שלנו. חלק מהדברים אנחנו כבר מזהים. בתוך FileSystem יש גישה למערכות הקבצים שלנו.

ה-Registry היא מבנה נתונים שמשמש לשמירת הגדרות של תוכנות שונות במחשב. לדוגמה, פיתחו את תוכנת אקסל. הקטינו את החלון שלה והזיזו אותו למקום לבחירתכם. סיגרו את החלון. כעת פיתחו שנית את תוכנת האקסל- המיקום וגודל החלון נותרו זהים. זאת מכיוון שהמידע נשמר ב-Registry.

כל רשומה ב-Registry כוללת מפתח (Key) וערך (Value).
כעת נלמד ליצור ערכים ולשנות ערכים ברגיסטרי.

הריצו את הפקודה `Get-PSProvider` ובחנו את השורה של ה-`Registry`. באמצעות Powershell כל המידע משוקף לנו במבנה של קבצים ותיקיות, ממש כאילו היינו מתבוננים בדיסק הקשיח. ניתן לראות שמבחינת Powershell ישנן שתי מערכות קבצים- `HKCU`, `HKLM`. האותיות `CU` מציינות "Current User", כלומר הגדרות ספציפיות למשתמש הנוכחי, כיוון שיכולים להיות מספר משתמשים על מחשב אחד. האותיות `LM` מציינות "Local Machine", כלומר כל מה ששייך למחשב עצמו ומשותף לכל מי שמשתמש בו.

חישבו, על סמך מה שראינו עד כה, איך ניתן להכנס לתוך מערכת הקבצים של ה-`Registry`?

נכון, באמצעות פקודת `cd`. לדוגמה:

```
PS C:\> cd hkcu:
```

```
PS HKCU:\> cd hk\m:
```

```
PS HKLM:\>
```

כעת נרצה ליצור רשומה חדשה ברגיסטרי. הרשומה תמוקם בתת התיקיה system\HKLM, שמה יהיה "bla" והערך שלה יהיה "blabla".

כיצד ניתן ליצור ערך חדש? פקודת New-Item, תוכלו למצוא עליה פרטים ב-help.

```
PS HKCU:\> New-Item -path ".\system" -Name "bla" -value "blabla"
```

```
Hive: HKEY_CURRENT_USER\system
```

Name	Property
----	-----
bla	(default) : blabla

השתמשו בפקודת Remove-Item, כרגיל ה-help הוא חברכם הטוב. זהירות! אתם עלולים למחוק דברים חשובים, לכן בסוף הפקודה הוסיפו את הסיומת "WhatIf". סיומת זו תאפשר לכם לראות מה היו קורה אילו הייתם מריצים את הפקודה, אך מבלי להריץ אותה למעשה. לדוגמה, אם נבקש לסגור את כל הProcessים ששמן chrome נקבל הודעות כגון אלו:

```
PS C:\> stop-process -Name chrome -WhatIf
what if: Performing the operation "Stop-Process" on target "chrome (1812)".
what if: Performing the operation "Stop-Process" on target "chrome (4872)".
```

ה- Environment, או כפי שהוא נכתב env, כולל את מה שנקרא משתני הסביבה. מהם? ראינו שיש ב-Powershell משתנים "רגילים", שמגדירים עם הסימן \$ בתחילתם. משתני הסביבה שונים מהם.

משתני הסביבה משמשים להגדרות שונות שמסייעות הן להרצה של תוכניות והן להקל על חיי המשתמש במחשב. עיברו ל-env (פשוט על ידי הפקודה cd:env) והתרשמו מהשמות של משתני הסביבה. הם די מסבירים את עצמם. לדוגמה, יש משתנה סביבה שכולל את מספר המעבדים שיש למחשב ומשתנה אחר שכולל מידע על סוג המעבד. מי שמתכנת תוכנה כלשהי יכול להניח שהמידע הזה ימצא במשתני הסביבה, מה שיכול להקל עליו את עבודתו. משתני סביבה נוספים מסייעים למשתמש לעבוד בנוחות במחשב. לדוגמה, משתנה סביבה בשם PATH כולל את אוסף הספריות שהמחשב מצפה למצוא בהן קבצי הרצה, כך שכל תוכנה שמוגדרת ב-PATH ניתנת להרצה מכל תיקיה בה אנו נמצאים. משתמש יכול לשנות את משתני הסביבה בקלות, כפי שנראה מיד.

משתני הסביבה מחולקים לשני סוגים- של המשתמש ושל המערכת כולה. או כפי שהם מוגדרים, user ו-system. משתני סביבה של המשתמש קובעים דברים ספציפיים למשתמש. לדוגמה, התיקיה בה יישמרו קבצים זמניים. אלו קבצים ששייכים למשתמש ספציפי, ועלולה להיות בעיה לחלוק אותם על כל המשתמשים במחשב. לעומת זאת, משתנה של ה-system משפיע על כל התוכנות שמריצים כל המשתמשים.

לסיכום, משתנה סביבה מוגדר על ידי 3 ערכים:

1. השם של משתנה הסביבה- רצוי שזה יהיה שם בעל משמעות, שמסביר מה הוא עושה

2. הערך שניתן למשתנה הסביבה

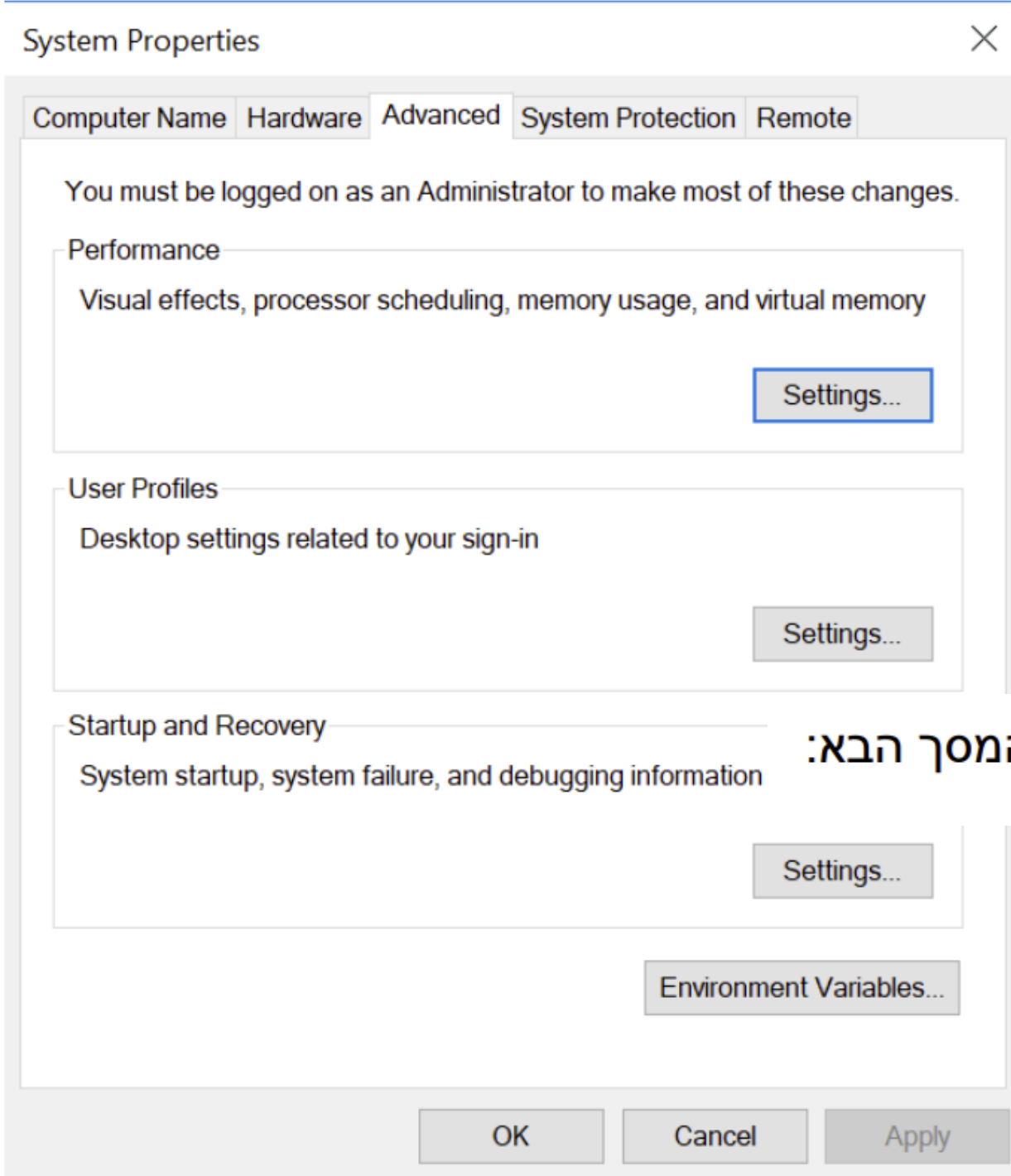
3. מי מכיר את משתנה הסביבה. האפשרויות הן:

i. משתנה הסביבה מוכר רק על ידי התוכנה שהגדירה אותו ורק כל עוד התוכנה רצה

ii. כל התוכנות שמריץ המשתמש שהגדיר את משתנה הסביבה. אפשרות זו נקראת User

iii. כל התוכנות במחשב, אפשרות זו נקראת System

כדי להגיע לממשק ידני של עריכת משתני הסביבה, נקיש בשורת החיפוש של Windows את המילה "environment" ונבחר באפשרות "Edit the System Environment Variables".



נקליק על הכפתור "Environment Variables" ונגיע אל המסך הבא:

User variables for barak

Variable	Value
OneDrive	C:\Users\barak\OneDrive
Path	C:\Users\barak\AppData\Local\Microsoft\WindowsApps;
TEMP	C:\Users\barak\AppData\Local\Temp
TMP	C:\Users\barak\AppData\Local\Temp

New...

Edit...

Delete

System variables

Variable	Value
NUMBER_OF_PROCESSORS	8
OS	Windows_NT
Path	C:\Program Files (x86)\Intel\iCLS Client\;C:\Program Files\Intel\iCL...
PATHEXT	.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC
PROCESSOR_ARCHITECTURE	AMD64
PROCESSOR_IDENTIFIER	Intel64 Family 6 Model 142 Stepping 10, GenuineIntel
PROCESSOR_LEVEL	6
PROCESSOR_REVISION	8e0a

New...

Edit...

Delete

מיותר לציין שכל מה שקשור למשתני סביבה ניתן לביצוע באמצעות פקודות Powershell. יצירת משתנה סביבה היא פשוטה ביותר. הפקודה נראית בדיוק כמו יצירת משתנה רגיל, פרט לכך שלפני שם המשתנה יש env:, המציין שזהו משתנה סביבה. לדוגמה:

```
PS C:\> $env:name = "Shooki"
```

כעת אם נבצע dir, או Get-ChildItem, נראה את המשתנה החדש שנוצר.

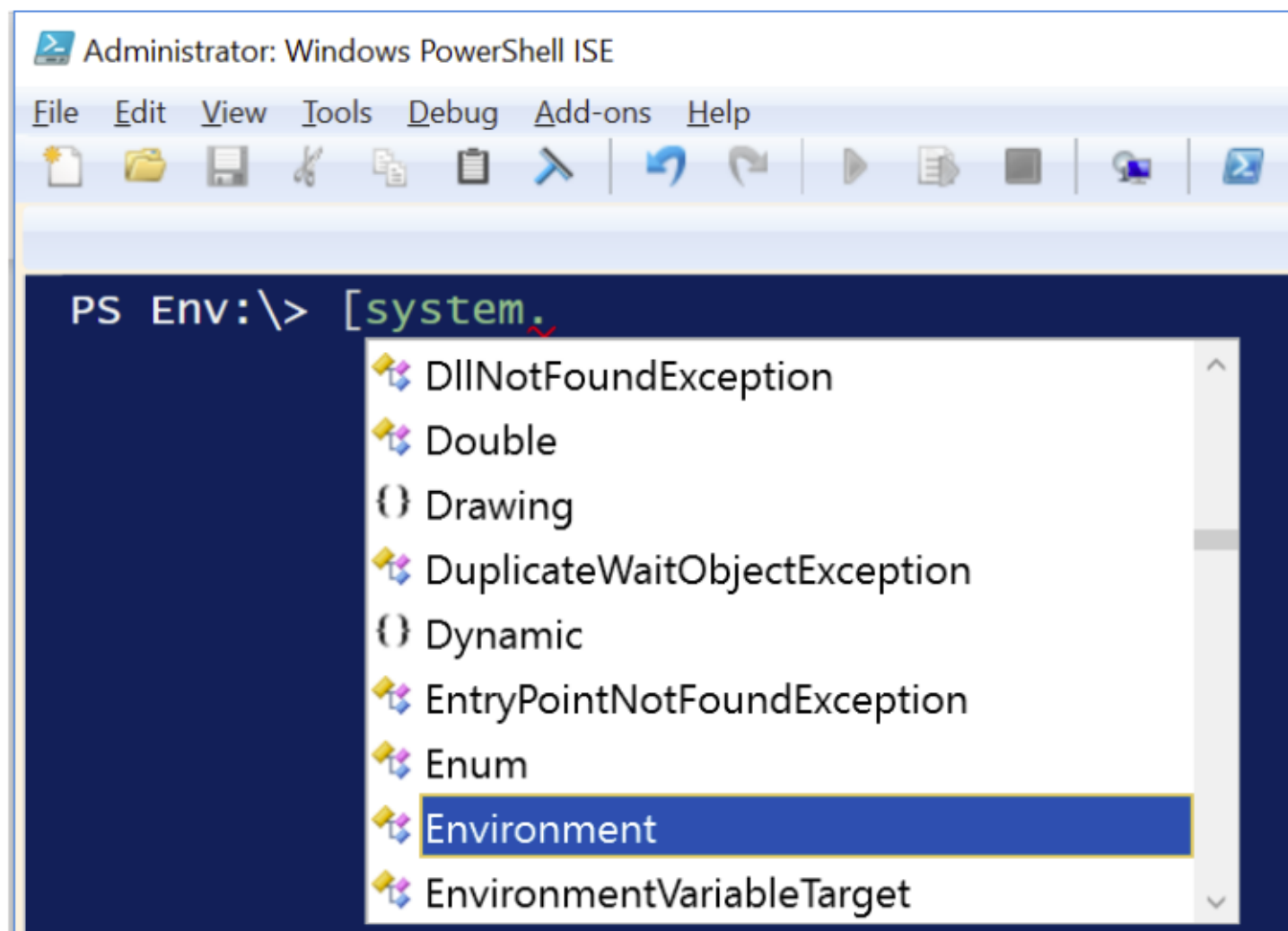
עם זאת, שימו לב, המשתנה החדש מוגדר רק בתוך Powershell וכל עוד לא סגרנו אותו. זאת מכיוון שלא הגדרנו מי מכיר את משתנה הסביבה. אופס! 😊

כדי לעשות זאת, נשתמש בפונקציות של WinAPI.

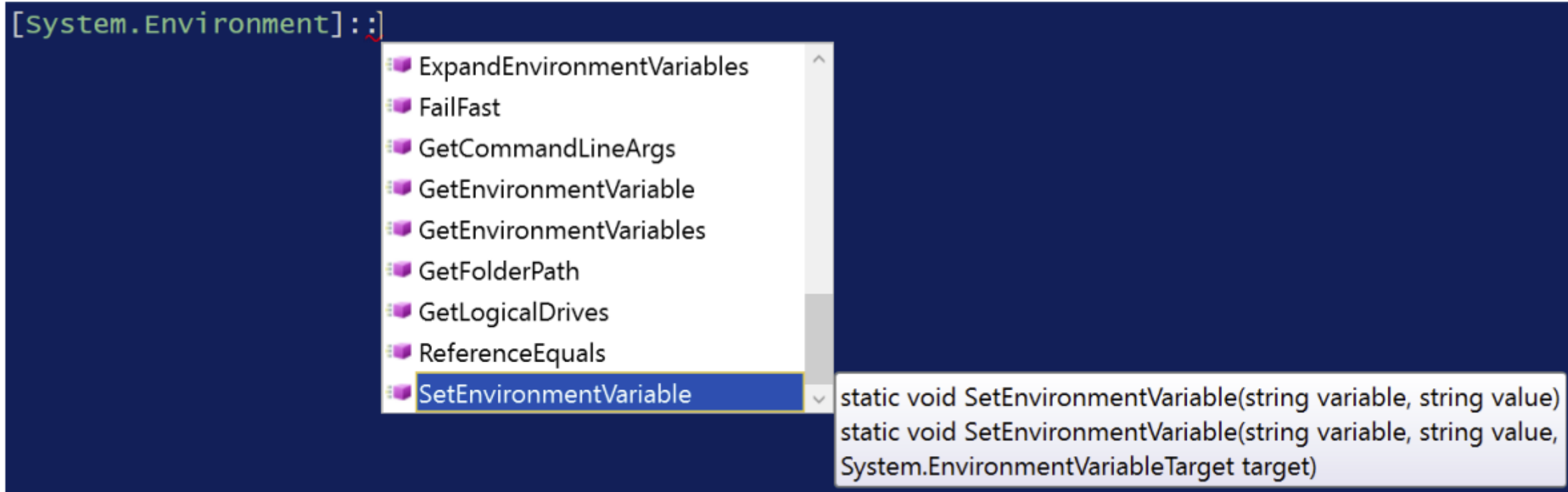
עכשיו כאשר למדנו על משתני סביבה, נרצה להוסיף משתנה סביבה כלשהו. הסיבה שנרצה לבצע זאת דרך משתנה סביבה היא מכיוון שכדי לבצע פעולות אוטומטיות, באמצעות סקריפטים, יש צורך בשימוש בקוד. הדרך הידנית הנוחה באמצעות מסך המשתמש של Windows מתאימה לשינוי חד פעמי.

המשימה שלנו היא להוסיף משתנה סביבה בשם STAM, שהערך שלו יהיה "123", והוא יהיה משתנה שיוכר על ידי ה-User בלבד. נתחיל.

נתחיל בלפתוח סוגריים מרובעים, ולאחריהם נכתוב system ואז נקודה. עורך ה-Powershell ייתן לנו לבחור מכל המחלקות האפשריות של WinAPI. כאשר נדפדף ביניהן, נמצא שיש מחלקה שנקראת Environment- כיוון שאנחנו רוצים לבצע פעולות על משתני סביבה, זה מה שאנחנו זקוקים לו.



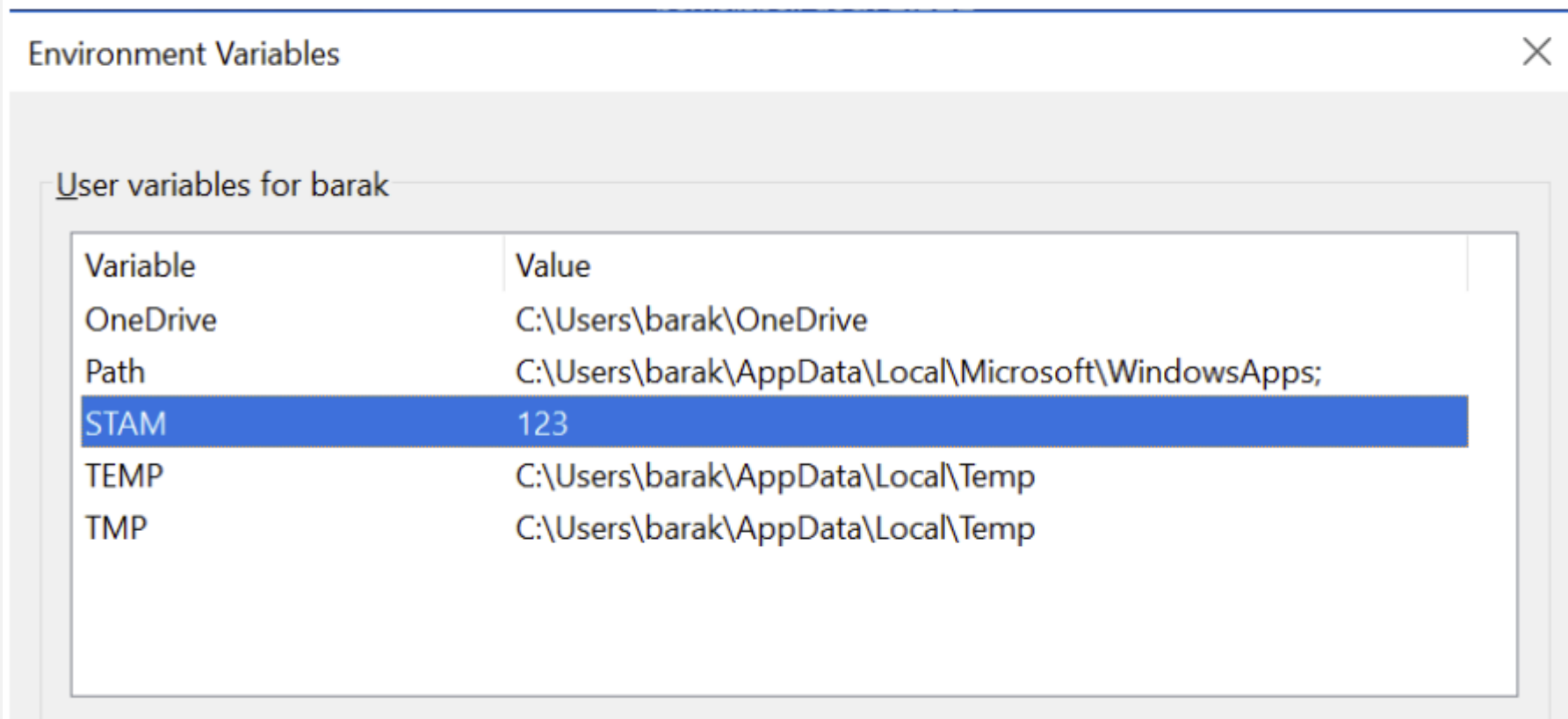
נסגור את הסוגריים המרובעים, ונכתוב :: (פעמיים נקודותיים). הנקודותיים הכפולים אומרים שאנחנו רוצים להשתמש בפונקציה מתוך מחלקה שבחרנו. כרגיל, Powershell יציג בפנינו את כל הפונקציות האפשריות. נדפדף בהן עד שנגיע לפונקציה המבוקשת-



בריבוע שצץ לנו מצד ימין מופיעים כל הפרמטרים שהפונקציה SetEnvironmentVariable מקבלת. אפשר לראות שיש לה שתי גרסאות- גרסה אחת שמקבלת שני פרמטרים, וגרסה אחרת שמקבלת שלושה. כאשר מתכנתים מחלקה שכוללת פונקציות, ניתן לכתוב כמה גרסאות של פונקציה בעלת אותו שם אך פרמטרים שונים. דבר זה נקרא OverLoading. רק הגרסה שמקבלת שלושה פרמטרים כוללת אפשרות לקבוע את ה-target, כלומר מי מכיר את משתנה הסביבה. לכן נשתמש בגרסה זו:

```
PS Env:\> [System.Environment]::SetEnvironmentVariable("STAM", "123", "USER")
```

לאחר הרצת שורת הקוד, נוכל לצפות בשינוי שחל במשתני הסביבה של ה-user שלנו:



באופן טבעי, חלק מכתובת קוד הוא פקודות שכוללות תנאים. כך גם ב-Powershell, ניתן לכתוב פקודות IF-ELSE.

```
PS C:\WINDOWS\system32> Help about_if -ShowWindow
```

ABOUT_IF

Short description

Describes a language command you can use to run statement lists based on the results of one or more conditional tests.

Examples:

The simplest if statement contains a single command and doesn't contain any elseif statements or any else statements. The following example shows the simplest form of the if statement:

```
if ($a -gt 2) {  
    Write-Host "The value $a is greater than 2."  
}
```

Syntax

The following example shows the if statement syntax:

```
if (<test1>)  
    {<statement list 1>}  
[elseif (<test2>)  
    {<statement list 2>}]  
[else  
    {<statement list 3>}]
```

תוך כדי הקריאה, תבחינו בכך שישנם סימנים שונים שאינם מוכרים לכם. לדוגמה `lt`, `ge`, `eq`. אל דאגה, כיתבו את הפקודה הבאה-

Help about_comparison_operators

ABOUT_COMPARISON_OPERATORS

Short description

The comparison operators in PowerShell can either compare two values or filter elements of a collection against an input value.

Long description

Comparison operators let you compare values or finding values that match specified patterns. PowerShell includes the following comparison operators:

EQUALITY

- `-eq`, `-ieq`, `-ceq` - equals
- `-ne`, `-ine`, `-cne` - not equals
- `-gt`, `-igt`, `-cgt` - greater than
- `-ge`, `-ige`, `-cge` - greater than or equal
- `-lt`, `-ilt`, `-clt` - less than
- `-le`, `-ile`, `-cle` - less than or equal

כיתבו סקריפט שקולט ערך מהמשתמש. אם הערך גדול מ-5, הדפסו "too high". אם הערך קטן מ-5 הדפסו "too low", אחרת הדפסו "high five!".

```
1  $num = Read-Host "Enter a number"
2
3  if ($num -gt 5) {
4      Write-Host "too high"
5  }
6  elseif ($num -lt 5) {
7      Write-Host "too low"
8  }
9  else {
10     Write-Host "high five!"
11 }
12
13
```

אחד הדברים הנחמדים והשימושיים ב-Unix הוא ה-Pipes, ולא פלא ש-Microsoft אימצה את הרעיון לתוך Powershell. הרעיון הוא פשוט- לשרשר פקודות, כך שהפלט של פקודה אחת הופך להיות הקלט של הפקודה הבאה. הסימן שאומר לשרשר פקודות הוא "|", הקו האנכי, שנמצא מעל מקש ה-enter. מה יכול להיות מועיל בשרשור פקודות? נמחיש את היתרון של Pipes באמצעות הפקודה המוכרת Get-Process.

הדבר הראשון שנרצה לעשות הוא לשמור את התוצאות לקובץ. קל מאד. ב-Powershell ישנן פקודות שמתחילות ב-export ומאפשרות לשמור לקבצים בכל מיני פורמטים. נבחר בפורמט csv הדומה לתוכנת excel:

```
PS C:\> Get-Process | Export-Csv -Path C:\Users\barak\procs.csv
```

הדבר הראשון שנרצה לעשות הוא לשמור את התוצאות לקובץ. קל מאד. ב-Powershell ישנן פקודות שמתחילות ב-export ומאפשרות לשמור לקבצים בכל מיני פורמטים. נבחר בפורמט csv הדומה לתוכנת excel:

```
PS C:\> Get-Process | Export-Csv -Path C:\Users\barak\procs.csv
```

AutoSave Off

procs • Saved to this PC

Search

FileHomeInsertDrawPage LayoutFormulasDataReviewViewAutomateHelp

CommentsShare

A1fx#TYPE System.Diagnostics.Process

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
1	#TYPE Sys	em.Diagnostics.Process																								
2	Name	SI	Handles	VM	WS	PM	NPM	Path	Company	CPU	FileVersion	ProductVe	Descriptio	Product	__NounNe	BasePrior	ExitCode	HasExited	ExitTime	Handle	SafeHandl	HandleCo	Id	MachineN	MainWinc	Main
3	Aggregato	0	182	2.2E+12	14327808	3309568	10680	C:\WINDO	Microsoft	2.484375	10.0.2610	10.0.2610	Microsoft	Microsoft	Process	8		FALSE		5236	Microsoft	182	6560	.		0
4	ai	1	226	2.2E+12	28237824	41975808	16184	C:\Progra	Microsoft	0.4375	0.20.47.0	0.20.47.0	Local AI H	Local AI	Process	8		FALSE		5132	Microsoft	226	10488	.		0
5	aimgr	1	379	2.2E+12	34197504	45965312	21416	C:\Progra	Microsoft	0.21875	0.20.47.0	0.20.47.0	Local AI M	Local AI	Process	8		FALSE		6064	Microsoft	379	1656	.		0
6	AppAction	1	395	2.2E+12	47198208	12308480	21008	C:\WINDO	Microsoft	7.546875	10.0.2610	10.0.2610	AppAction	Microsoft	Process	8		FALSE		5156	Microsoft	395	12996	.		0
7	Applicatio	1	547	2.2E+12	55877632	56295424	28096	C:\WINDO	Microsoft	1.796875	10.0.2610	10.0.2610	Applicatio	Microsoft	Process	8		FALSE		4912	Microsoft	547	6712	.	262892	Settir
8	audiodg	0	236	2.2E+12	18825216	8200192	12032	C:\WINDO	Microsoft	0.25	10.0.2610	10.0.2610	Windows	Microsoft	Process	8		FALSE		6312	Microsoft	236	14132	.		0
9	backgrou	1	331	2.2E+12	2535424	19066880	28688	C:\WINDO	Microsoft	0.140625	10.0.2610	10.0.2610	Backgrou	Microsoft	Process	8		FALSE		6260	Microsoft	331	3340	.		0
10	backgrou	1	436	2.2E+12	52449280	26820608	20520	C:\WINDO	Microsoft	0.8125	10.0.2610	10.0.2610	Backgrou	Microsoft	Process	8		FALSE		4280	Microsoft	436	8500	.		0
11	backgrou	1	331	2.2E+12	2301952	19009536	28688	C:\WINDO	Microsoft	0.1875	10.0.2610	10.0.2610	Backgrou	Microsoft	Process	8		FALSE		6088	Microsoft	331	10760	.		0
12	backgrou	1	318	2.2E+12	3256320	15175680	16304	C:\WINDO	Microsoft	0.09375	10.0.2610	10.0.2610	Backgrou	Microsoft	Process	8		FALSE		5212	Microsoft	318	14148	.		0
13	conhost	1	99	2.2E+12	7147520	1183744	7888	C:\WINDO	Microsoft	0	10.0.2610	10.0.2610	Console V	Microsoft	Process	8		FALSE		6248	Microsoft	99	2908	.		0
14	conhost	1	99	2.2E+12	6983680	1200128	7888	C:\WINDO	Microsoft	0.03125	10.0.2610	10.0.2610	Console V	Microsoft	Process	8		FALSE		5196	Microsoft	99	5908	.		0
15	conhost	1	122	2.2E+12	9732096	1605632	10472	C:\WINDO	Microsoft	0.015625	10.0.2610	10.0.2610	Console V	Microsoft	Process	8		FALSE		5316	Microsoft	122	13128	.		0
16	CrossDevi	1	569	2.2E+12	64151552	54079488	28640	C:\WINDO	Microsoft	3.015625	2126.8501	2126.8501.0.0		Microsoft	Process	8		FALSE		1560	Microsoft	569	3828	.		0
17	CrossDevi	1	929	2.2E+12	1.03E+08	31059968	79832	C:\Progra	Microsoft	5.1875	1.26032.8	1.26032.8	Microsoft	CrossDevi	Process	8		FALSE		6416	Microsoft	929	7596	.		0
18	csrss	1	770	2.2E+12	8261632	3772416	25352			166.7031					Process	13						770	904	.		0
19	csrss	0	716	2.2E+12	7987200	2465792	24272			17.6875					Process	13						716	976	.		0
20	ctfmon	1	522	2.2E+12	33521664	6000640	19640	C:\WINDO	Microsoft	5.328125	10.0.2610	10.0.2610	CTF Load	Microsoft	Process	13		FALSE		6384	Microsoft	522	3396	.		0
21	DataExch	1	342	2.2E+12	39714816	11337728	21904	C:\Windov	Microsoft	1.125	10.0.2610	10.0.2610	Data Exch	Microsoft	Process	10		FALSE		5664	Microsoft	342	11148	.		0

באיזה תרחיש פעולה כזו יכולה להיות שימושית? לדוגמה, כדי לבדוק שינויים בProcessים. ייתכן שאנחנו חושדים שתוכנה זדונית כלשהי מופעלת לעיתים במחשב שלנו ומבצעת דברים בלא ידיעתנו. נבצע שמירה של כל הProcessים ברגע מסויים, ונבדוק את השינויים. כדי לבדוק את השינויים, נבצע Get-Process פעם נוספת ונשווה את התוצאות לקובץ ששמרנו. רגע אחד! אנחנו ממש לא מתכוונים לעבור אחד אחד על כל הProcessים, יש שיטה מגניבה לבצע את זה... הכירו את הפקודה diff, שהיא ה-alias של Compare-Object.

הפרמטר הראשון שניתן ל-diff הוא ReferenceObject, כלומר הבסיס אליו משווים. הפרמטר השני יהיה DifferenceObject, כלומר המצב לאחר השינוי. לכן נרצה שאחד הפרמטרים יקבל את כל ה-Processes הקיימים כרגע ואחד הפרמטרים יקרא את כל ה-Processes שנשמרו בקובץ:

```
PS C:\users\barak> diff -ReferenceObject (Get-Process) `
>> -DifferenceObject (Import-Csv -Path .\procs.csv)
```

הסבר: לאחר הפקודה diff יופיע הפרמטר Reference שאומר "הנה הבסיס". הערך שיהיה בתוכו הוא התוצאה של Get-Process כרגע. לאחר מכן ישנו הפרמטר Difference, שאומר "הנה השינוי", והוא מקבל את אותו הקובץ ששמרנו קודם לכן.

```
PS C:\users\barak> diff -ReferenceObject (Get-Process) `
>> -DifferenceObject (Import-Csv -Path .\procs.csv)
```

הערה חשובה: כפי שאתם רואים הקוד מחולק לשתי שורות. מה שמאפשר לעשות זאת הוא סימן ה"טיק", שנמצא מעל מקש הטאב (על אותו מקש נמצא גם סימן ה- "~"). סימן זה אינו מוכר על ידי סביבת הדיבוג שעבדנו איתה עד כה, Powershell_ise, אך הוא מוכר על ידי Powershell ה"רגיל". אפשר לכתוב את אותה פקודה בלי סימן ה"טיק", בשורה אחת יחידה, אך כדי לשפר את הקריאות של צילום הקוד הוא מוצג לפניכם בצורה מפוצלת.

אם תריצו את הקוד שכתבנו כעת, תראו איך המסך שלכם מוצף במידע. זאת מכיוון שכל שינוי קל יוצג בו. לדוגמה, שינוי בניצול המעבד או בניצול הזיכרון של Process מסויים. כיוון שכל הזמן יש שינויים קטנים, יהיו המון הבדלים. אנחנו מעוניינים לבדוק רק אילו Process ים השתנו, מעניין אותנו רק השמות של הProcess ים. נעשה ניסוי קטן- פיתחו את notepad (אנו מניחים שהוא היה סגור עד כה. אם הוא היה פתוח- סיגרו אותו, העיקר שמצבו ישתנה בין סגור/פתוח לטובת הניסוי שלנו). כעת הריצו את הפקודה אך עם הוראה להשוות רק את השם של הProcess:

```
PS C:\users\barak> diff -ReferenceObject (Get-Process) `
>> -DifferenceObject (Import-Csv -Path .\procs.csv) -Property name

name                               SideIndicator
----                               -
notepad                           =>
```

והנה, מצאנו מיד את השם של הProcess שאינו זהה בין המצב הקודם למצב הנוכחי.

מיד נראה דברים רבים נוספים שניתן לעשות עם Pipes, אבל לפני כן נצטרך ללמוד מהם אובייקטים.

בחלק הקודם יצרנו קובץ csv של כל ה-Processes שלנו. הקליקו עליו וצפו בו. מיד תשימו לב לכך שיש בו אינספור עמודות. כל שורה מייצגת Process וכל עמודה מייצגת מידע כלשהו שקשור ל-Process הזה. אנחנו מבינים מכך ש-Powershell שומר לכל Process מבנה נתונים שכולל את כל המידע עליו.

לא משנה איזו שפת תכנות למדתם, פייתון, Java או C#, לבטח מוכר לכם המושג "מחלקה", או Class. זהו מבנה נתונים הכולל אוסף של משתנים ופונקציות. המשתנים קרויים "Properties" והפונקציות שמשוייכות למחלקה קרויות "Methods". מכל מחלקה ניתן ליצור אובייקטים שיהיו להם את כל המשתנים והפונקציות המוגדרים במחלקה.

עד כה עבדנו עם אובייקטים של Powershell לא מעט, אך פשוט לא קראנו להם ככה.

```
PS C:\> notepad
```

```
PS C:\> $proc = Get-Process -name notepad
```

```
PS C:\> $proc
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
-----	-----	-----	-----	-----	--	--	-----
237	14	3104	13212	0.03	30644	1	notepad

```
PS C:\> $proc | Get-Member
```

כאשר אנחנו מבצעים `Get-Process`, אנחנו מקבלים את המידע על כל האובייקטים מסוג `Process`. כל שורה היא אובייקט שהוא `Process`. זה פיצ'ר נחמד מאד, בהשוואה ללינוקס ששם כל שורה היא מחרוזת. אנחנו רוצים להתמקד באובייקט אחד, ב-`Process` ספציפי, כדי לראות מה האוסף של כל ה-`Properties` וה-`Methods` שיש לו, ואילו עוד דברים משוייכים אליו. הפקודה `Get-Member` נותנת לנו את כל מה שמשוייך לאובייקט. בטח שמתם לב שכדי להשתמש בפקודה זו, אנחנו מעבירים אליה את האובייקט באמצעות `Pipe`. אגב, לפקודה `Get-Member` יש `alias` שנקרא `gm`, נשתמש בו מעכשיו.

קיראו את התוצאות שיש להרצת הפקודה האחרונה. האם אתם רואים מהיכן הגיע כל המידע ששמור בקובץ ה-`csv` שצפיתם בו בתחילת חלק לימוד זה?

בתחילת התוצאות של `gm`, כתוב מה שם האובייקט המדובר. במקרה שלנו:

```
TypeName: System.Diagnostics.Process
```

בתחילת התוצאות של gm, כתוב מה שם האובייקט המדובר. במקרה שלנו:

```
TypeName: System.Diagnostics.Process
```

לאחר מכן ישנה רשימה של Properties. בואו נשתמש בהם! נבצע Get-Process, אך נציג רק את השם ואת הזיכרון RAM שבשימוש:

```
PS C:\> Get-Process | select name, ws
```

Name	WS
----	--
aesm_service	5550080
ApplicationFrameHost	37777408
audiodg	20299776

יפה. ומה אם אנחנו רוצים לבצע מיון לפי Property כלשהו? לדוגמה לפי צריכת זכרון ה-RAM? פשוט מאד, נעשה לאובייקט Pipe לתוך הפונקציה Sort-Object, שניתן גם לקרוא לה sort. כיוון שאנחנו רוצים למיין מהגבוה לנמוך, נפעיל את הפרמטר descending:

```
PS C:\> Get-Process | Sort-Object ws -Descending
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
-----	-----	-----	-----	-----	--	--	-----
0	0	1884	773504	254.09	3332	0	Memory Compression
756	61	385568	366076	124.52	28624	1	powershell_ise
1262	510	314304	355724	240.56	34628	1	SkypeApp
837	113	368632	328508	485.25	2568	1	chrome

נתקדם עוד צעד, לפקודה מורכבת יותר. הפעם נשרשר Pipe לתוך Pipe. אנחנו רוצים להדפיס רק את 10 ה-Processes בעלי צריכת הזיכרון RAM הגבוהה ביותר.

```
PS C:\> Get-Process | sort-object WS -Descending | select-object -First 10
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
-----	-----	-----	-----	-----	--	--	-----
0	0	1884	722260	254.28	3332	0	Memory Compression
2329	47	478696	384460	731.00	26508	1	chrome
927	67	393732	383424	129.28	28624	1	powershell_ise
862	119	374724	370516	504.78	2568	1	chrome
1231	518	314788	355824	248.92	34628	1	skypeApp
2089	3221	657012	280556	1,647.28	2356	0	vsserv
445	100	270116	263304	786.95	29848	1	chrome
2773	91	311560	261612	1,261.63	27908	1	chrome
541	67	205332	217704	322.88	28204	1	spotify
2123	457	266356	189052	2,656.20	26124	1	WINWORD

כמו שאולי ניחשתם כל הזמן, לכל הפקודות הללו יש alias, אך לא השתמשנו בהם עד כה כדי לא להקשות על קריאת הקוד. הנה אותה הפקודה בצורה מקוצרת:

```
PS C:\> gps | sort WS -Descending | select -First 10
```

נמשיך עוד שלב בסולם ה-Pipes. מדוע לא ננסה לעשות משהו עם ה-Process? ראינו כי ל-Process יש מתודה בשם kill, שהיא ה-alias של Stop-Process. אולי נבדוק מה יקרה אם נסגור את ה-Process הללו? כיוון שפעולה זו עלולה להסתיים במסך כחול ומחשב תקוע, נשתמש בפרמטר whatif:

```
PS C:\> gps | sort ws -Descending | select -First 10 | kill -whatif
what if: Performing the operation "Stop-Process" on target "Memory Compression (3332)".
what if: Performing the operation "Stop-Process" on target "chrome (2568)".
what if: Performing the operation "Stop-Process" on target "powershell_ise (28624)".
what if: Performing the operation "Stop-Process" on target "SkypeApp (34628)".
what if: Performing the operation "Stop-Process" on target "vsserv (2356)".
what if: Performing the operation "Stop-Process" on target "chrome (29848)".
what if: Performing the operation "Stop-Process" on target "chrome (27908)".
what if: Performing the operation "Stop-Process" on target "chrome (26508)".
what if: Performing the operation "Stop-Process" on target "spotify (28204)".
what if: Performing the operation "Stop-Process" on target "WINWORD (26124)".
```

צורה נוספת לעבודה עם אובייקטים היא שימוש ב-Where-Object. פעולה זו מאפשרת מעבר על כל האובייקטים וביצוע תנאים עליהם. נמחיש זאת. נניח שאנחנו רוצים לצפות בכל ה-Processes שה-PID שלהם (כלומר המספר המזהה של ה-Process) הוא בתחום של 0 עד 1000:

```
PS C:\> gps | where-object {$_.Id -lt 1000}
```

נסביר. ה-Pipe מעביר הלאה את כל האובייקטים שיצאו מ-gps. כל אובייקט כזה נלקח על ידי הפקודה Where-Object ומוכנס לתוך הסוגריים המסולסלים, במקום בו נמצא \$_. כל Process שעובר את התנאי-במקרה זה שה-ID שלו הוא בעל מספר קטן (less than) 1000, עובר הלאה לשלב הבא- הדפסה למסך.

ביצוע Invoke-Expression למחרוזת

זהו, אנחנו כמעט בסוף. הדבר האחרון אותו נלמד, הוא פקודה חשובה מאד בהיבטים של נוזקות. פקודת Invoke-Expression, שה-alias שלה הוא IEX, מקבלת מחרוזת ומריצה אותה כפי שמריצים קוד Powershell רגיל. המשמעות היא שאנחנו יכולים להגדיר משתנים מטיפוס מחרוזת ולהריץ אותם. הנה דוגמה:

```
PS C:\> $command = "dir"  
PS C:\> iex $command
```

למה להגביל את עצמנו לפקודה קצרה כל כך?

```
PS C:\> $command = "get-service | select name"
```

```
PS C:\> iex $command
```

בואו נתפרע מעט. מדוע לא ניצור מחרוזת בתוך משתנה סביבה ונריץ אותו?

```
PS C:\> $env:command = "write-host 'hello'"
```

```
PS C:\> iex $env:command  
hello
```

שימו לב לשילוב של מרכאות כפולות עם מרכאות בודדות, שנועד לאפשר לנו להגדיר מחרוזת בתוך מחרוזת.

הרצת קוד זדוני מתוך משתנה סביבה היא אפיק פעולה שנוזקות שמחות לנצל, מכיוון שמשתנה הסביבה נשאר תמיד בזיכרון ועשוי להשפיע על כל המשתמשים במחשב, אם משתנה הסביבה מוגדר כ-System.